

重庆大学计算机专业核心课程《机器学习》

第5章 神经网络

授课教师：郑林江

第5章 神经网络

➔ 5.1 神经元模型

5.2 感知机与多层网络

5.3 误差逆传播算法

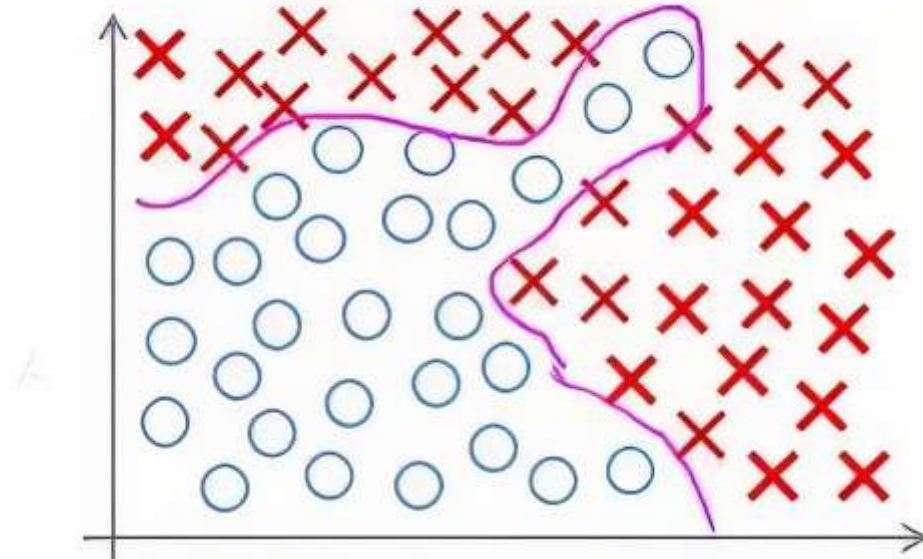
5.4 全局最小与局部极小

5.5 其他常见神经网络

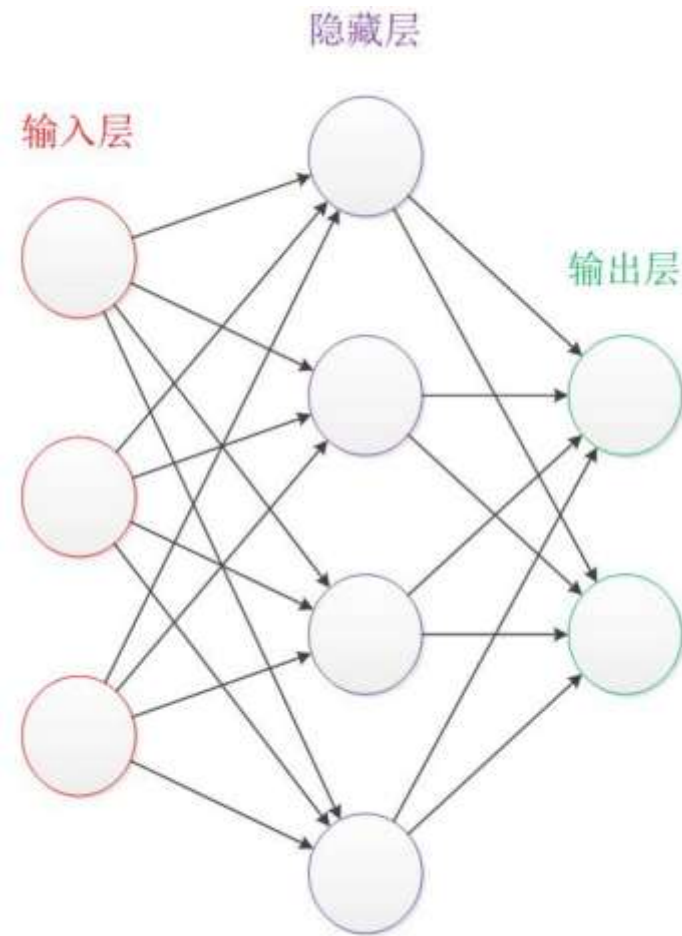
5.6 深度学习

- 对于非线性的分类问题，在特征量较少的情况，可以用多项式类型的Logistic回归或其它算法来处理。但是一旦特征量较多，多项式的Logistic回归就会很困难。

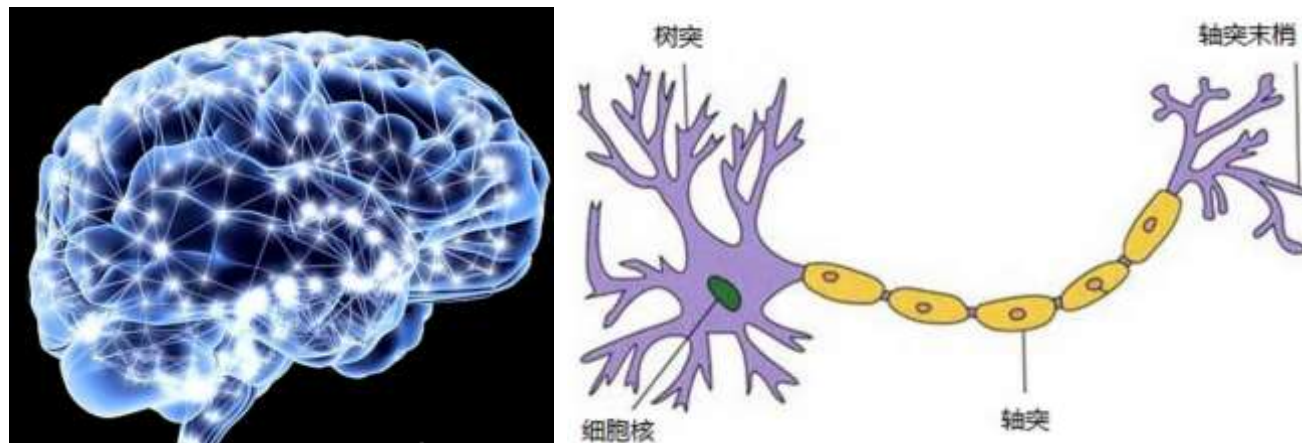
Non-linear Classification



- 设计一个神经网络时，输入层与输出层的节点数往往是固定的(分别是特征数和目标类别数)，中间层则可以自由指定(自己设计)；
- 结构图里的关键不是圆圈（代表“**神经元**”），而是连接线（代表“神经元”之间的连接）。每个连接线对应一个不同的权重（其值称为权值），需要训练得到。



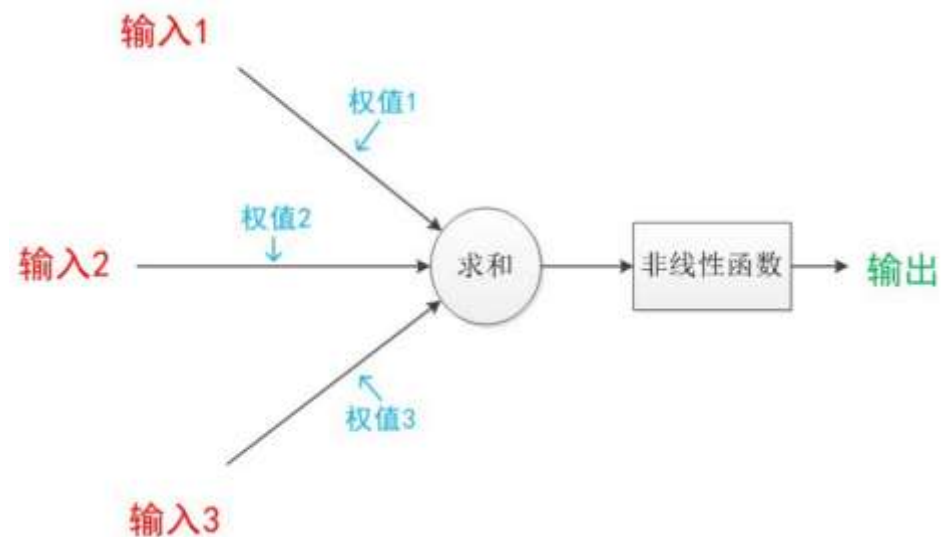
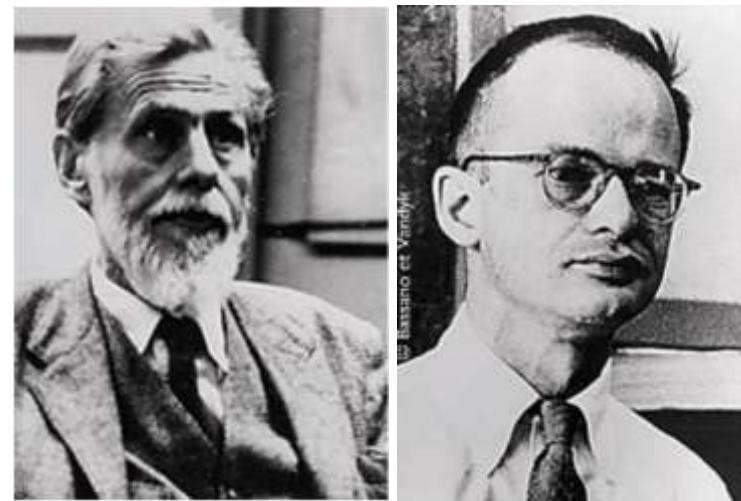
- 神经网络是一种**模拟大脑**的算法。神经网络是由具有适应性的**简单单元**组成的广泛并行互连的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。
- 神经网络中最基本的成分是**神经元模型(Neural Model)**(又称Logistic unit)。对于人类而言，视觉听觉是由大脑的神经中枢产生。而**神经中枢**是由大量的神经元相互连接而成。一个神经元通过树突接受其他神经元传来的化学物质（信息），从而改变该神经元的电位，当电位到达某一阈值(threshold)时，该神经元被激活，即“兴奋”起来，从而通过轴突向其他神经元发送化学物质。



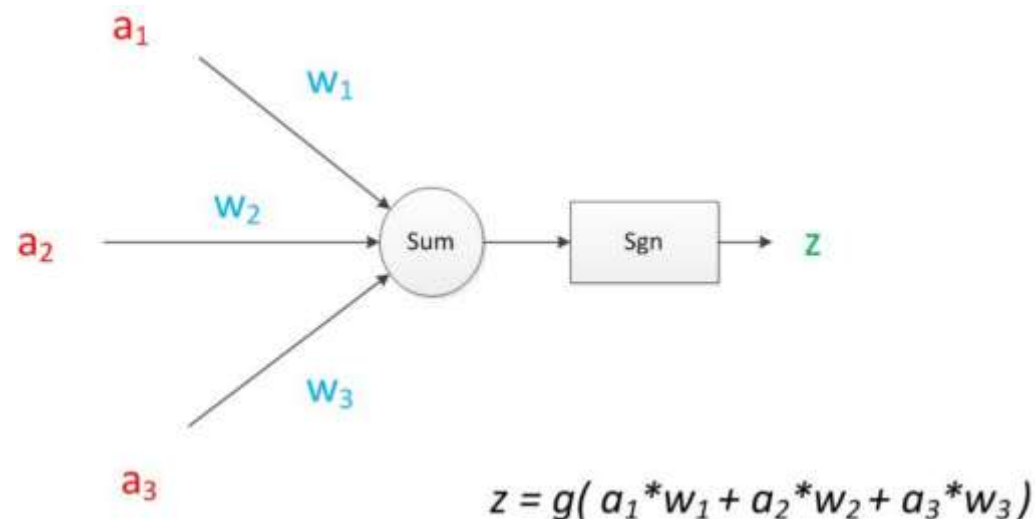
1. 外部刺激通过神经末梢，转化为电信号，转导到神经细胞（又叫神经元）。
2. 无数神经元构成神经中枢。
3. 神经中枢综合各种信号，做出判断。
4. 人体根据神经中枢的指令，对外部刺激做出反应。

□ 神经元模型

- 神经元模型(1943年发明)是一个包含输入，输出与计算功能的模型。输入可以类比为神经元的树突，而输出可以类比为神经元的轴突，计算则可以类比为细胞核。
- 一个典型的**神经元模型**：包含有3个输入，1个输出，以及2个计算功能。注意中间的箭头线。这些线称为“连接”。每个连接上有一个“权值”。

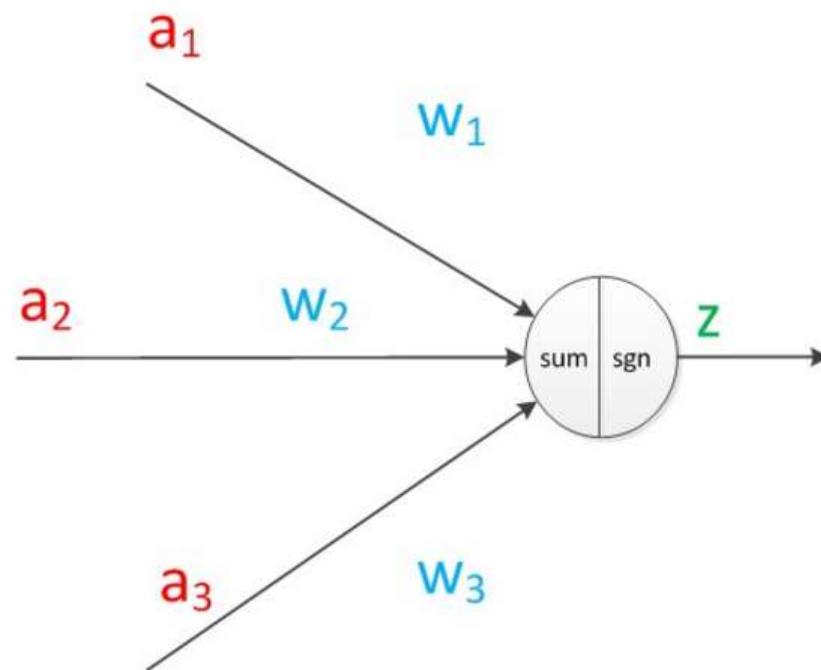


- 连接是神经元中最重要的东西。
每一个连接上都有一个**权重**。一个神经网络的**训练算法**就是让**权重的值调整到最佳**，以使得整个网络的预测效果最好(拟合)。
- 将神经元图中的所有变量用符号表示，写出输出的计算公式：
- z 是在输入和权值的线性加权和叠加了一个函数 g 的值。在MP（McCulloch and Pitts, 1943）模型里，函数 g 是sgn函数，也就是取符号函数。这个函数当输入大于0时，输出1，否则输出0。



□ 神经元模型

- 对神经元模型的图进行一些扩展。首先将sum函数与sgn函数合并到一个圆圈里，代表神经元的内部计算。其次，把输入a与输出z写到连接线的左上方，便于后面画复杂的网络。
- 神经元可以看作一个**计算与存储单元**。计算是神经元对其的输入进行计算功能。存储是神经元会暂存计算结果，并传递到下一层。



M-P 神经元模型 [McCulloch and Pitts, 1943]

- 输入：来自其他 n 个神经元传递过来的输入信号
- 处理：输入信号通过带权重的连接进行传递, 神经元接受到总输入值将与神经元的阈值进行比较
- 输出：通过激活函数的处理以得到输出

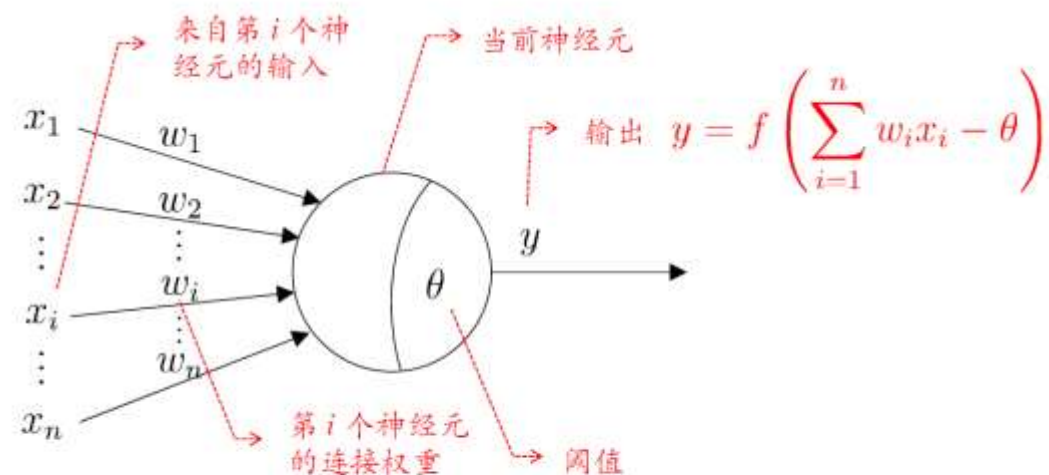


图 5.1 M-P 神经元模型

激活函数

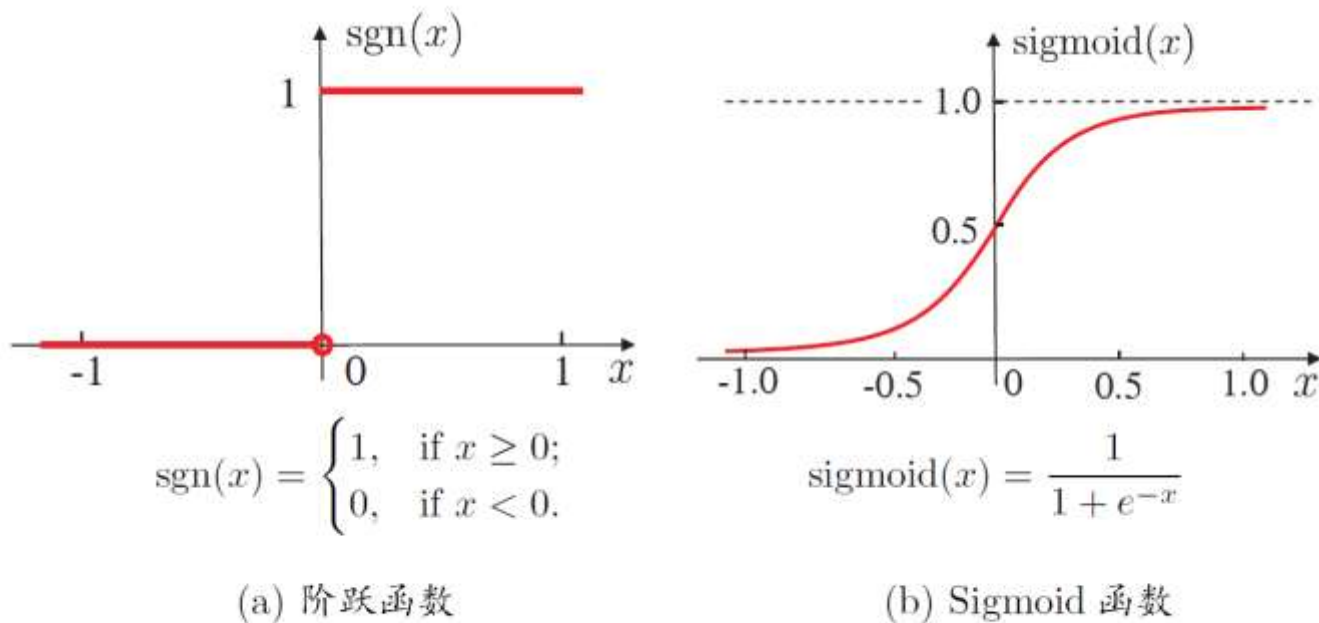


图 5.2 典型的神经元激活函数

- 理想激活函数是阶跃函数，0表示抑制神经元而1表示激活神经元。
- 阶跃函数具有不连续、不光滑等不好的性质，常用的是Sigmoid函数。

第5章 神经网络

5.1 神经元模型



5.2 感知机与多层网络

5.3 误差逆传播算法

5.4 全局最小与局部极小

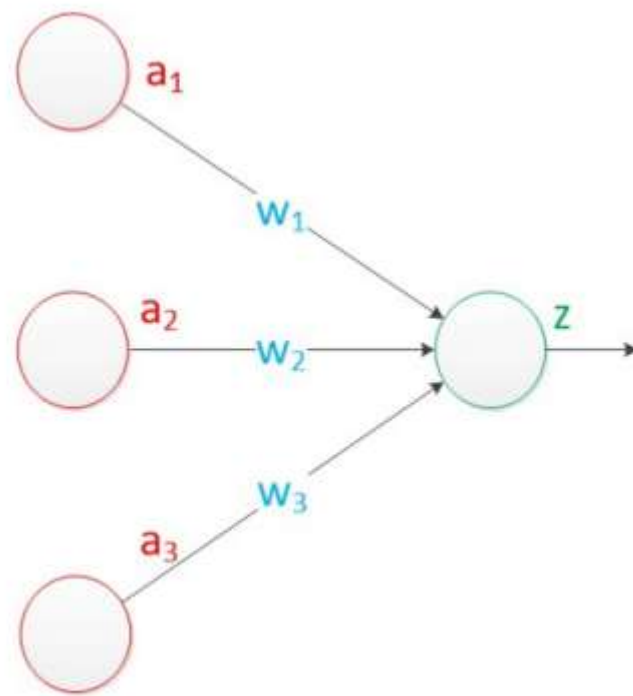
5.5 其他常见神经网络

5.6 深度学习

- 1958年，计算科学家Rosenblatt提出了由两层神经元组成的神经网络。他给它起了一个名字—“感知器”（**Perceptron**）（有的文献翻译成“感知机”）。
- 感知器是当时首个可以学习的人工神经网络。**Rosenblatt现场演示了其学习识别简单图像的过程，在当时的社会引起了轰动。**
- 人们认为已经发现了智能的奥秘，许多学者和科研机构纷纷投入到神经网络的研究中。美国军方大力资助了神经网络的研究，并认为神经网络比“原子弹工程”更重要。这段时间直到1969年才结束，**这个时期可以看作神经网络的第一次高潮。**

□ 感知机——单层神经网络

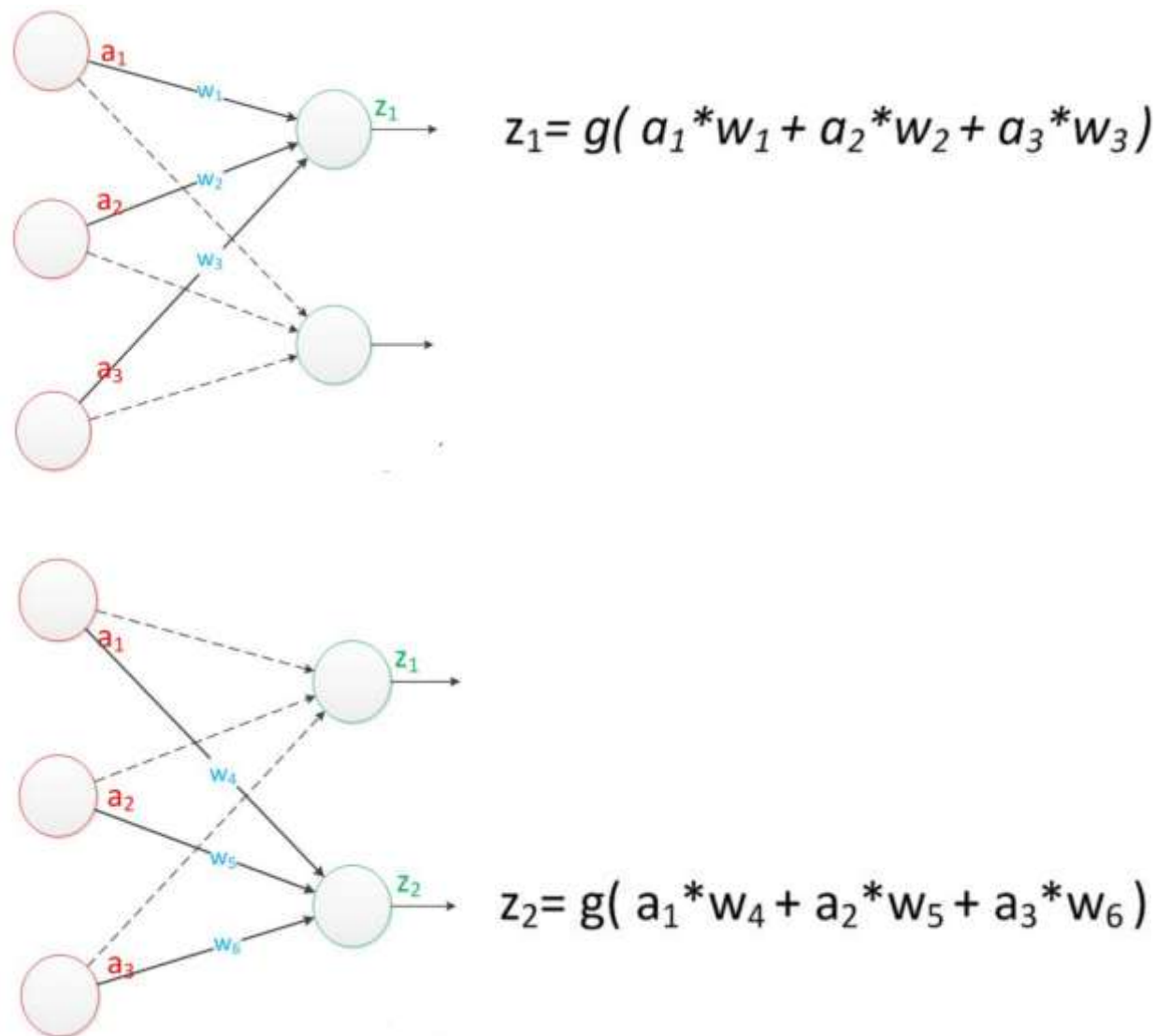
- 在原来MP模型的“输入”位置添加神经元节点，标志其为“输入单元”。其余不变。从本图开始，将权值 w_1, w_2, w_3 写到“连接线”的中间。
- 感知器有两个层次。分别是输入层和输出层。输入层里的“输入单元”只负责传输数据，不做计算。输出层里的“输出单元”则需要对前面一层的输入进行计算。此处只有一个计算层。



单层神经网络

□ 感知机——单层神经网络

- 假如要预测的目标不再是一个值，而是一个向量，例如[2,3]。那么可以在输出层再增加一个“输出单元”。（单分类变为多分类问题）
- 下图显示了带有两个输出单元的单层神经网络，其中输出单元 z_1 和 z_2 的计算公式如下图。



□ 感知机——单层神经网络

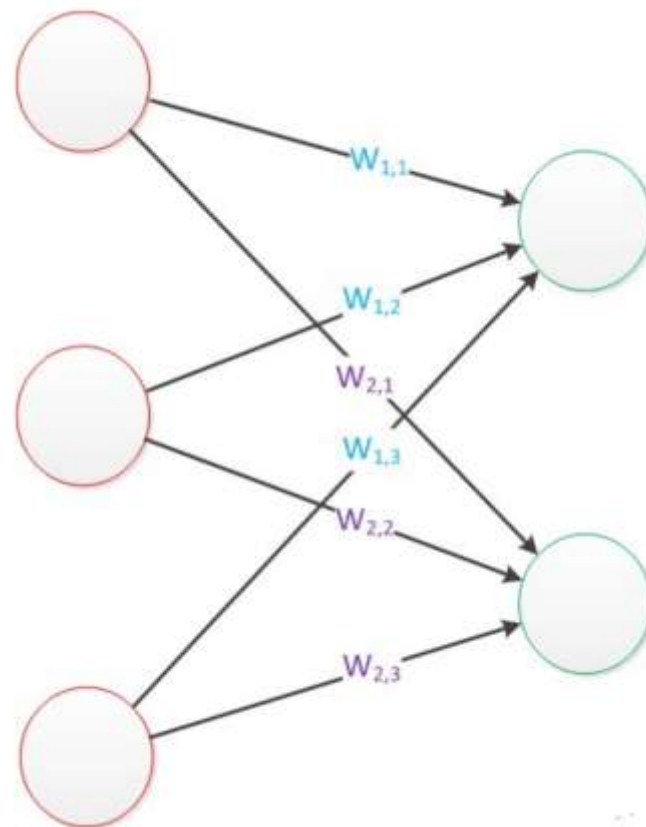
- 会发现这两个公式就是线性代数方程组。
因此可以用矩阵乘法来表达这两个公式。

$$z_1 = g(a_1 * w_{1,1} + a_2 * w_{1,2} + a_3 * w_{1,3})$$

$$z_2 = g(a_1 * w_{2,1} + a_2 * w_{2,2} + a_3 * w_{2,3})$$

$$g(\mathbf{W} * \mathbf{a}) = \mathbf{z};$$

$$[a_1, a_2, a_3]^T \quad [z_1, z_2]^T$$



- 感知机由两层神经元组成, 输入层接受外界输入信号传递给输出层, 输出层是**M-P**神经元 (阈值逻辑单元)
- 感知机能够容易地实现逻辑与、或、非运算
 - “与” $x_1 \wedge x_2$: 令 $w_1 = w_2 = 1, \theta = 0$, 则
 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$, 仅在 $x_1 = x_2 = 1$ 时, $y = 1$.
 - “或” $x_1 \vee x_2$: 令 $w_1 = w_2 = 1, \theta = 0.5$, 则
 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$, 仅在 $x_1 = 1$ 或者 $x_2 = 1$ 时, $y = 1$.
 - “非” $\neg x_1$: 令 $w_1 = -0.6, w_2 = 0, \theta = -0.5$, 则
当 $x_1 = 1$ 时, $y = 0$; 当 $x_1 = 0$, $y = 1$.

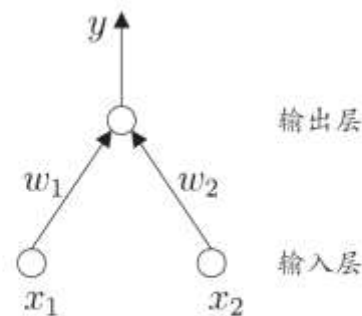


图 5.3 两个输入神经元的感知机网络结构示意图

感知机学习

■ 给定训练数据集, 权重 $w_i (i = 1, 2, \dots, n)$ 与阈值 θ 可以通过学习得到

■ 感知机学习规则

对训练样例 $(\mathbf{x}, y)$ 若当前感知机的输出为 $\hat{y}$, 则感知机权重调整规则为:

$$\begin{aligned} w_i &\leftarrow w_i + \Delta w_i \\ \Delta w_i &= \eta(y - \hat{y})x_i \end{aligned}$$

其中 $\eta \in (0, 1)$ 称为学习率

若感知机对训练样例 $(\mathbf{x}, y)$ 预测正确, 则感知机不发生变化; 否则根据错误程度进行权重的调整.

感知机求解异、或、非等问题

- 需注意的是，感知机只有输出层神经元进行激活函数处理，即只拥有一层功能神经元 (functional neuron), 其学习能力非常有限。事实上，上述与、或、非问题都是线性可分 (linearly separable) 的问题。
- 可以证明 [Minsky and Papert, 1969], 若两类模式是线性可分的, 即存在一个线性超平面能将它们分开, 如图5.4(a)-(c)所示, 则感知机的学习过程一定会收敛 (converge) 而求得适当的权向量 $w = (w_1; w_2; \dots; w_{n+1})$; 否则感知机学习过程将会发生振荡 (Huctuation), w 难以稳定下来, 不能求得合适解, 例如感知机甚至不能解决如图 5.4(d) 所示的 “非线性可分” 意味着用线性超平面无法划分。异或这样简单的非线性可分问题。

XOR Truth Table

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	1	0
1	0	1

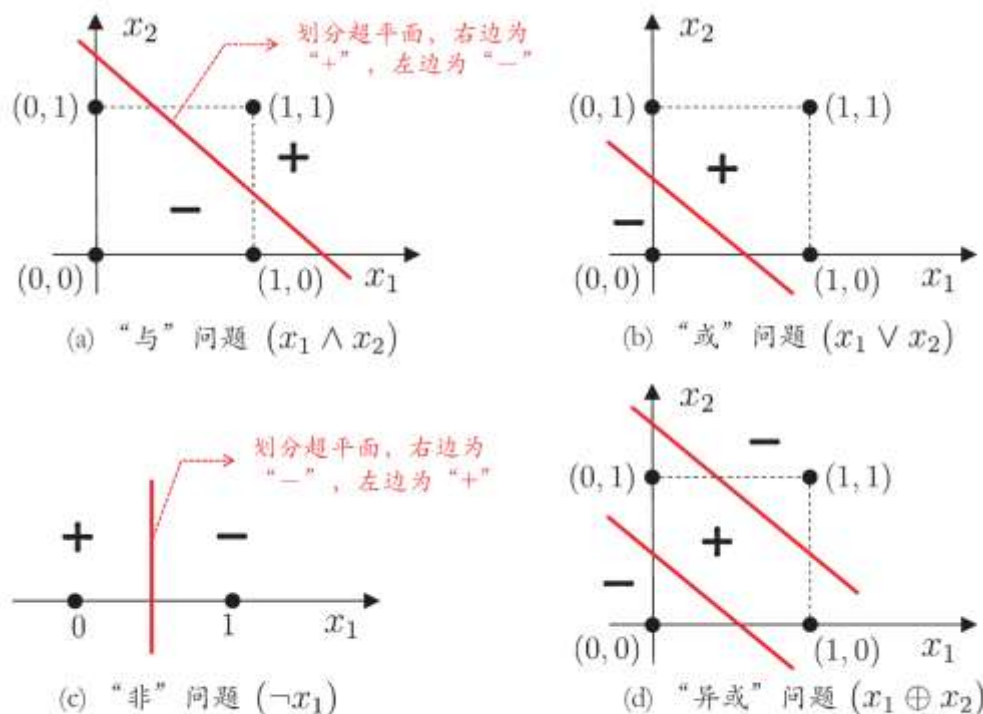


图 5.4 线性可分的“与”“或”“非”问题与非线性可分的“异或”问题

■ 感知机例子

- ✓ 城里正在举办一年一度的游戏动漫展览，小明拿不定主意，周末要不要去参观。他决定考虑三个因素。

1. 天气：周末是否晴天？
2. 同伴：能否找到人一起去？
3. 价格：门票是否可承受？



- ✓ 这就构成一个感知器。上面三个因素就是外部输入，最后的决定就是感知器的输出。如果三个因素都是 Yes（使用1表示），输出就是1（去参观）；如果都是 No（使用0表示），输出就是0（不去参观）。

■ 感知机例子

- ✓ 权重和阈值: 如果某些因素成立, 另一些因素不成立, 输出是什么? 比如, 周末是好天气, 门票也不贵, 但是小明找不到同伴, 他还要不要去参观呢?
- ✓ 现实中, 各种因素很少具有同等重要性: 某些因素是决定性因素, 另一些因素是次要因素。因此, 可以给这些因素指定权重 (weight), 代表它们不同的重要性。

- 天气: 权重为8
- 同伴: 权重为4
- 价格: 权重为4

天气是决定性因素, 同伴和价格都是次要因素。

■ 感知机例子

- ✓ 如果三个因素都为1，它们乘以权重的总和就是 $8 + 4 + 4 = 16$ 。如果天气和价格因素为1，同伴因素为0，总和就变为 $8 + 0 + 4 = 12$ 。
- ✓ 还需要指定一个阈值（threshold）。如果总和大于阈值，感知器输出1，否则输出0。假定阈值为8，那么 $12 > 8$ ，小明决定去参观。阈值的高低代表了意愿的强烈，阈值越低就表示越想去，越高就越不想去。决策过程使用数学表达如下。

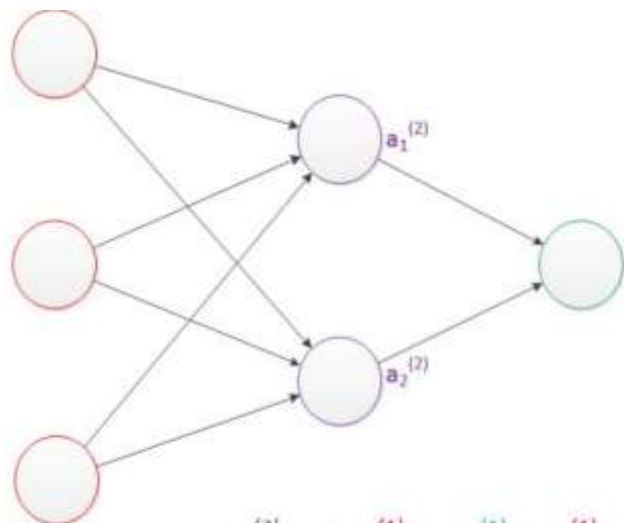
$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

- Minsky指出单层神经网络无法解决异或问题。但是当增加一个计算层以后，两层神经网络不仅可以解决异或问题，而且具有非常好的非线性分类效果。不过两层神经网络的计算是一个问题，没有一个较好的解法。
- 1986年，Rumelhar和Hinton等人提出了反向传播（Backpropagation, BP）算法，解决了两层神经网络所需要的复杂计算量问题，从而带动了业界使用两层神经网络研究的热潮。这时候的Hinton还很年轻，30年以后，正是他重新定义了神经网络，带来了神经网络复苏的又一春。
- LLM的例子（量变引起质变；因为相信所以看见）



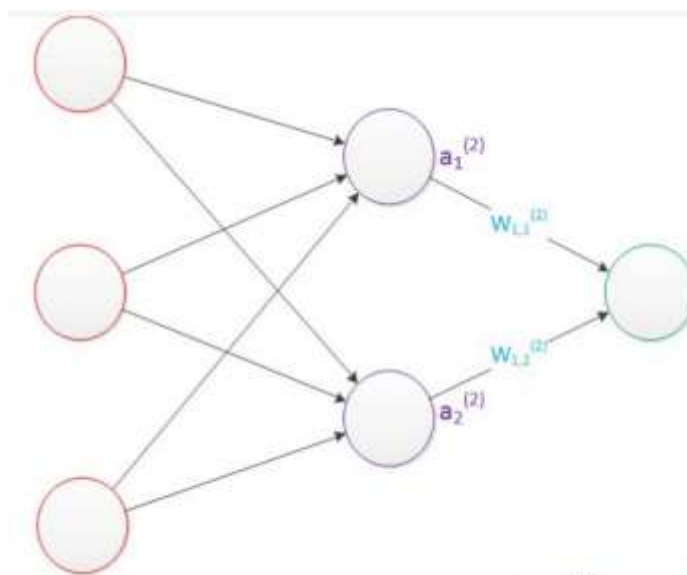
□ 两层神经网络——结构

- 两层神经网络除了包含一个输入层，一个输出层以外，还增加了一个中间层。



$$a_1^{(2)} = g(a_1^{(1)} * w_{1,1}^{(1)} + a_2^{(1)} * w_{1,2}^{(1)} + a_3^{(1)} * w_{1,3}^{(1)})$$

$$a_2^{(2)} = g(a_1^{(1)} * w_{2,1}^{(1)} + a_2^{(1)} * w_{2,2}^{(1)} + a_3^{(1)} * w_{2,3}^{(1)})$$



$$z = g(a_1^{(2)} * w_{1,1}^{(2)} + a_2^{(2)} * w_{1,2}^{(2)})$$

■ 多层感知机

- ✓ 解决异或问题的两层感知机

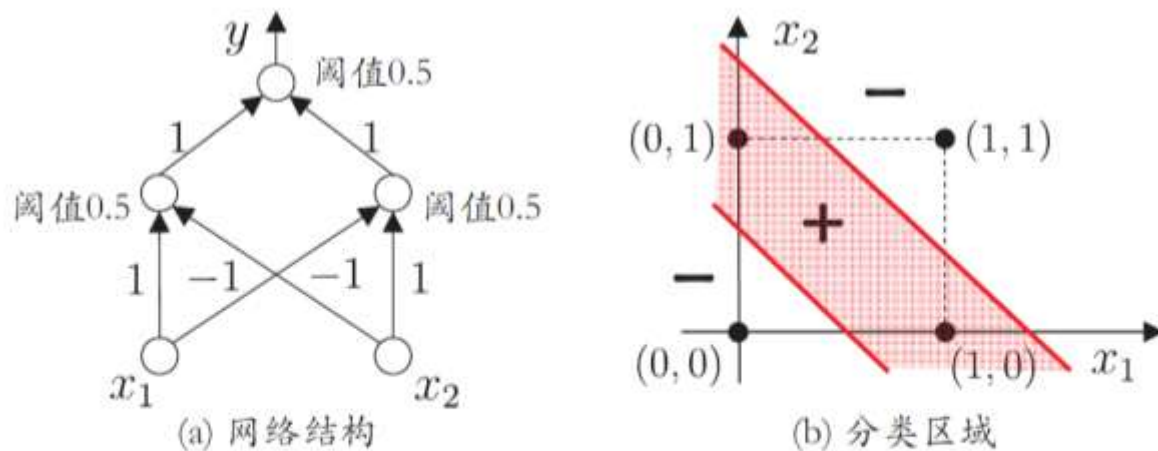
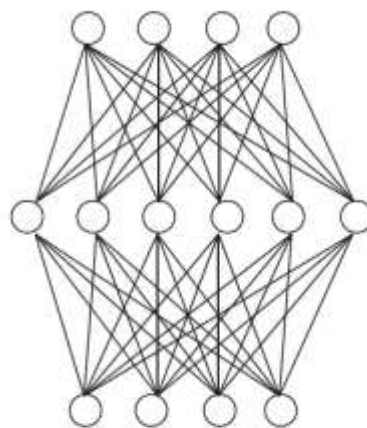


图 5.5 能解决异或问题的两层感知机

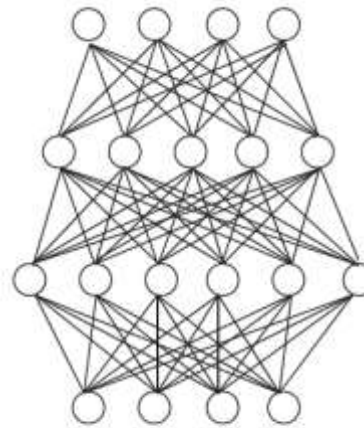
- ✓ 输出层与输入层之间的一层神经元, 被称之为**隐层或隐含层**, 隐含层和输出层神经元都是具有激活函数的功能神经元

■ 多层前馈神经网络

- ✓ **定义**：每层神经元与下一层神经元全互联，神经元之间不存在同层连接也不存在跨层连接
- ✓ **前馈**：输入层接受外界输入，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出
- ✓ **学习**：根据训练数据来调整神经元之间的“**连接权**”以及每个功能神经元的“**阈值**”
- ✓ **多层网络**：包含隐层的网络



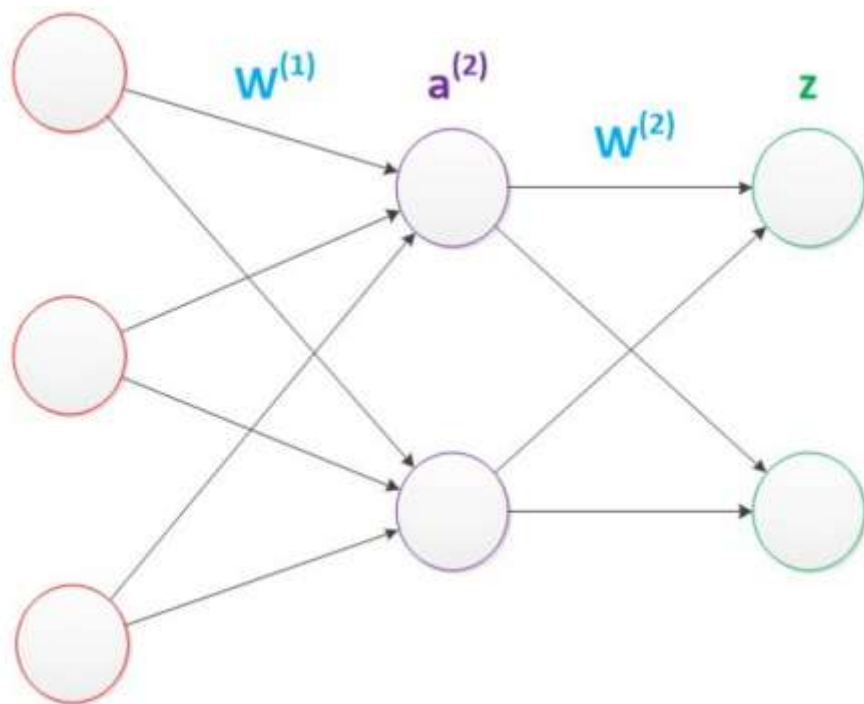
(a) 单隐层前馈网络



(b) 双隐层前馈网络

□ 两层神经网络——结构

- 两层神经网络除了包含一个输入层，一个输出层以外，还增加了一个中间层。



$$g(W^{(1)} * a^{(1)}) = a^{(2)};$$

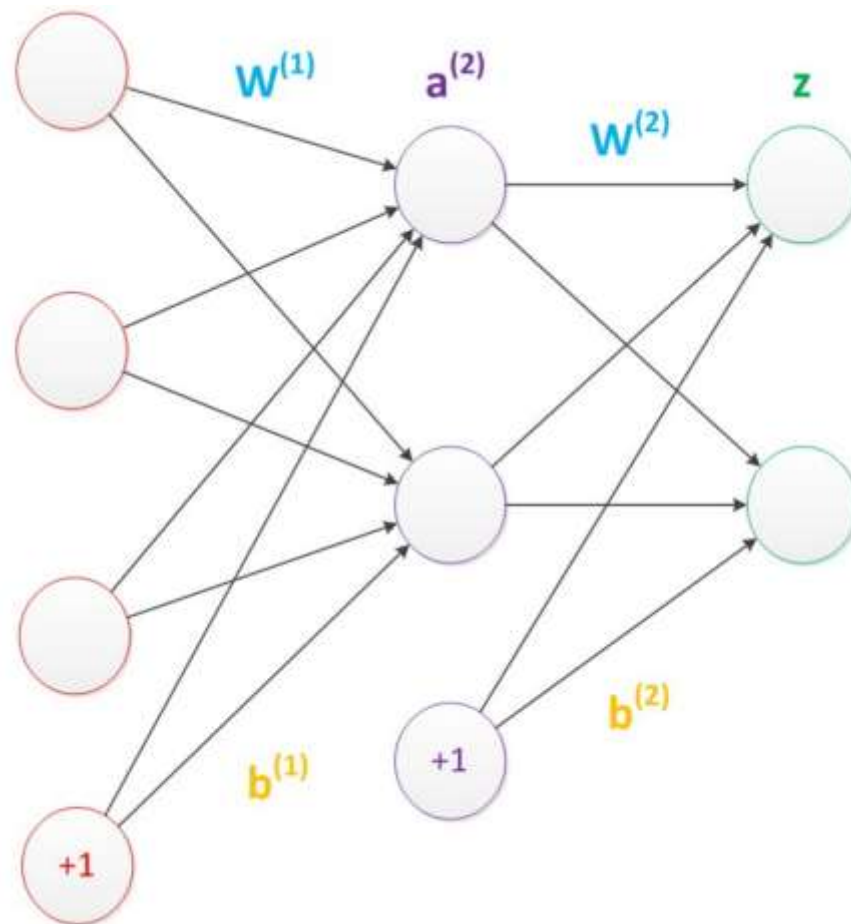
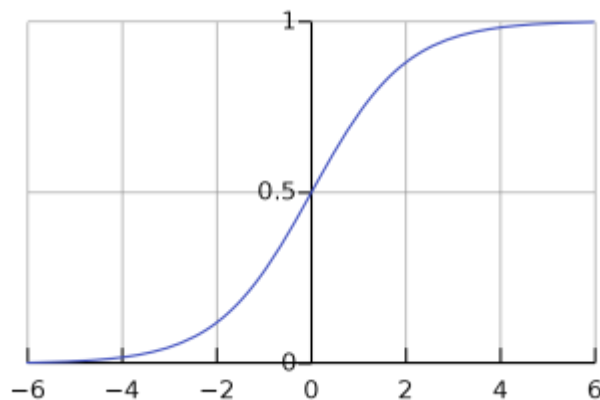
$$g(W^{(2)} * a^{(2)}) = z;$$

□ 两层神经网络——偏置

- 在两层神经网络中，不再使用sgn函数作为函数g，而是使用平滑函数**sigmoid**作为函数g。
函数g也称作**激活函数（active function）**（还是0到1，可微）。

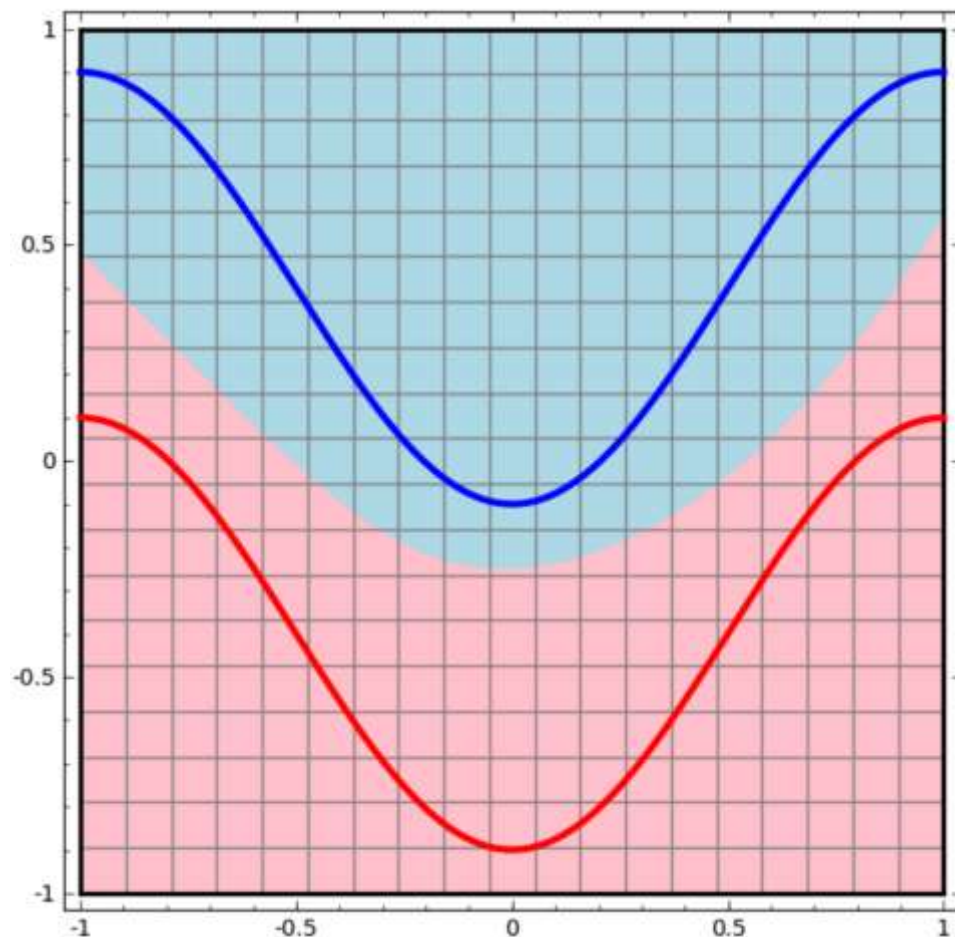
$$g(\mathbf{W}^{(1)} * \mathbf{a}^{(1)} + \mathbf{b}^{(1)}) = \mathbf{a}^{(2)};$$

$$g(\mathbf{W}^{(2)} * \mathbf{a}^{(2)} + \mathbf{b}^{(2)}) = \mathbf{z};$$



□ 两层神经网络——效果

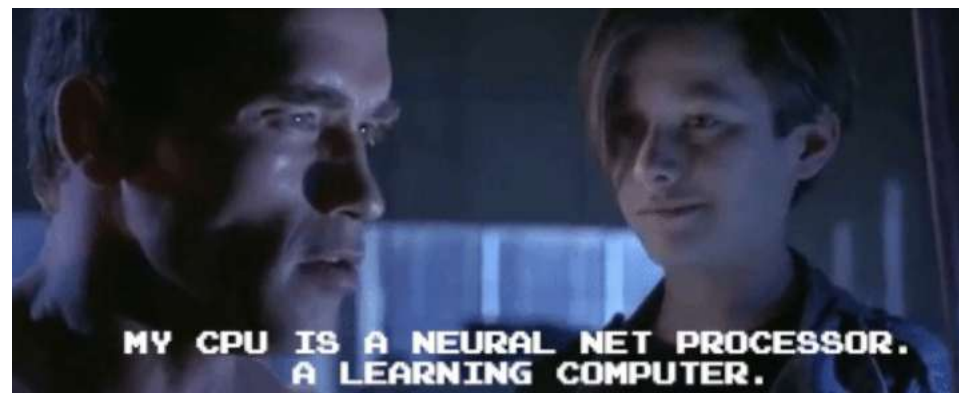
- 理论证明，两层神经网络可以无限逼近任意连续函数。也就是说，面对复杂的非线性分类任务，两层（带一个隐藏层）神经网络可以分类的很好。
- 例子：红色的线与蓝色的线代表数据。而红色区域和蓝色区域代表由神经网络划开的区域，两者的分界线就是决策分界。非常平滑，而且分类的很好。



□ 两层神经网络——思考

- 单层网络只能做线性分类任务。而两层神经网络中的后一层也是线性分类层，应该只能做线性分类任务。为什么两个线性分类任务结合就可以做非线性分类任务？
- 从输入层到隐藏层时，数据发生了空间变换。也就是说，两层神经网络中，隐藏层对原始的数据进行了一个空间变换，使其可以被线性分类，然后输出层的决策分界划出了一个线性分类分界线，对其进行分类。
- 两层神经网络可以做非线性分类的关键 - 隐藏层。矩阵和向量相乘，本质上就是对向量的坐标空间进行一个变换。因此，隐藏层的参数矩阵的作用就是使得数据的原始坐标空间从线性不可分，转换成了线性可分。两层神经网络通过两层的线性模型模拟了数据内真实的非线性函数。因此，多层的神经网络的本质就是复杂函数拟合。

- 两层神经网络在多个地方的应用说明了其效用与价值。
10年前困扰神经网络界的异或问题被轻松解决。神经网络在这个时候，已经可以发力于语音识别，图像识别，自动驾驶等多个领域。
- 历史总是惊人的相似，神经网络的学者们再次登上了《纽约时报》的专访。人们认为神经网络可以解决许多问题。就连娱乐界都开始受到了影响，当年的《终结者》电影中的阿诺都赶时髦地说一句：我的CPU是一个神经网络处理器，一个会学习的计算机。（1984）



- 但是神经网络仍然存在若干的问题：尽管使用了BP算法，一次神经网络的训练仍然耗时太久，而且困扰训练优化的一个问题就是局部最优解问题，这使得神经网络的优化较为困难。同时，隐藏层的节点数需要调参，这使得使用不太方便，工程和研究人員对此多有抱怨。
- 90年代中期，由Vapnik等人发明的SVM（Support Vector Machines，支持向量机）算法诞生，很快就在若干个方面体现出了对比神经网络的优势：无需调参；高效；全局最优解。基于以上种种理由，SVM迅速打败了神经网络算法成为主流。
- 神经网络的研究再次陷入了冰河期。当时，只要你的论文中包含神经网络相关的字眼，非常容易被会议和期刊拒收。



Vladimir Vapnik

- 在被人摒弃的10年中有几个学者仍然在坚持研究。代表人物是Geoffery Hinton教授。
- 2006年, Hinton在《Science》和相关期刊上发表了论文, 首次提出了“深度信念网络(DBN, Deep Belief Network)”的概念。与传统的训练方式不同, “深度信念网络”有一个“预训练”(pre-training)的过程, 这可以方便的让神经网络中的权值找到一个接近最优解的值, 之后再使用“微调”(fine-tuning)技术来对整个网络进行优化训练。这两个技术的运用大幅度减少了训练多层神经网络的时间。他给多层神经网络相关的学习方法赋予了一个新名词 - “深度学习”(Deep Learning)。

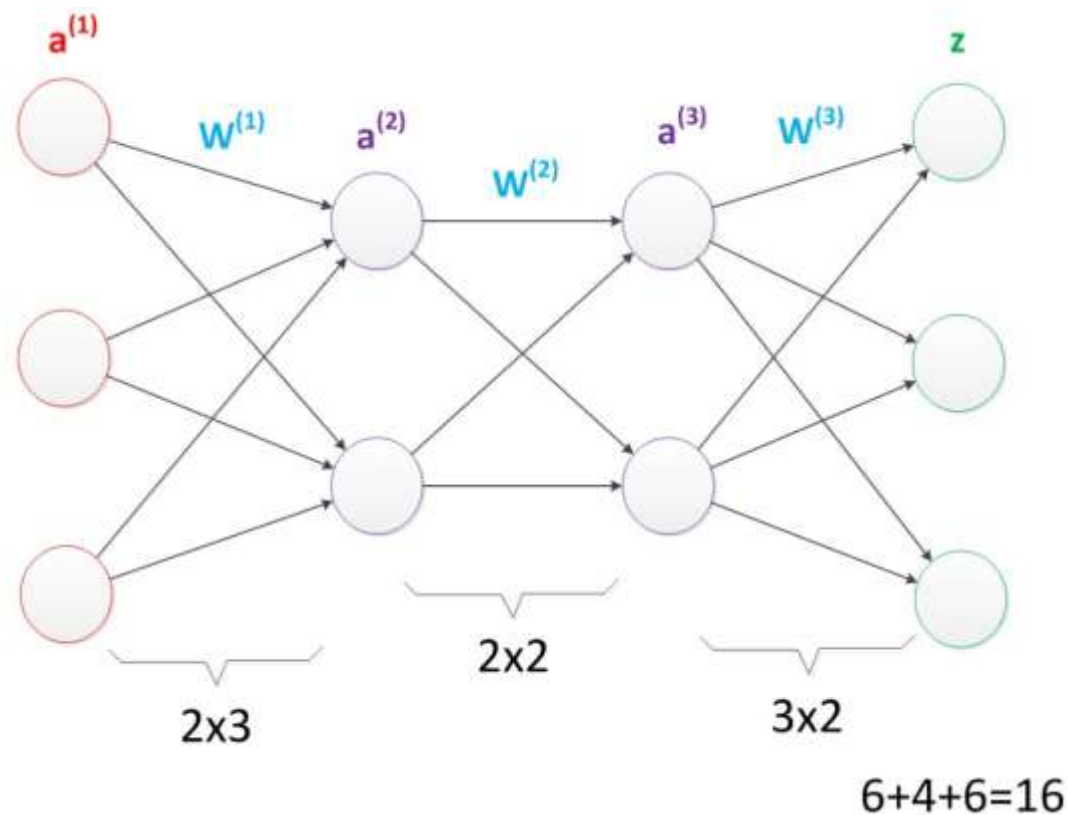


□ 多层神经网络—深度学习

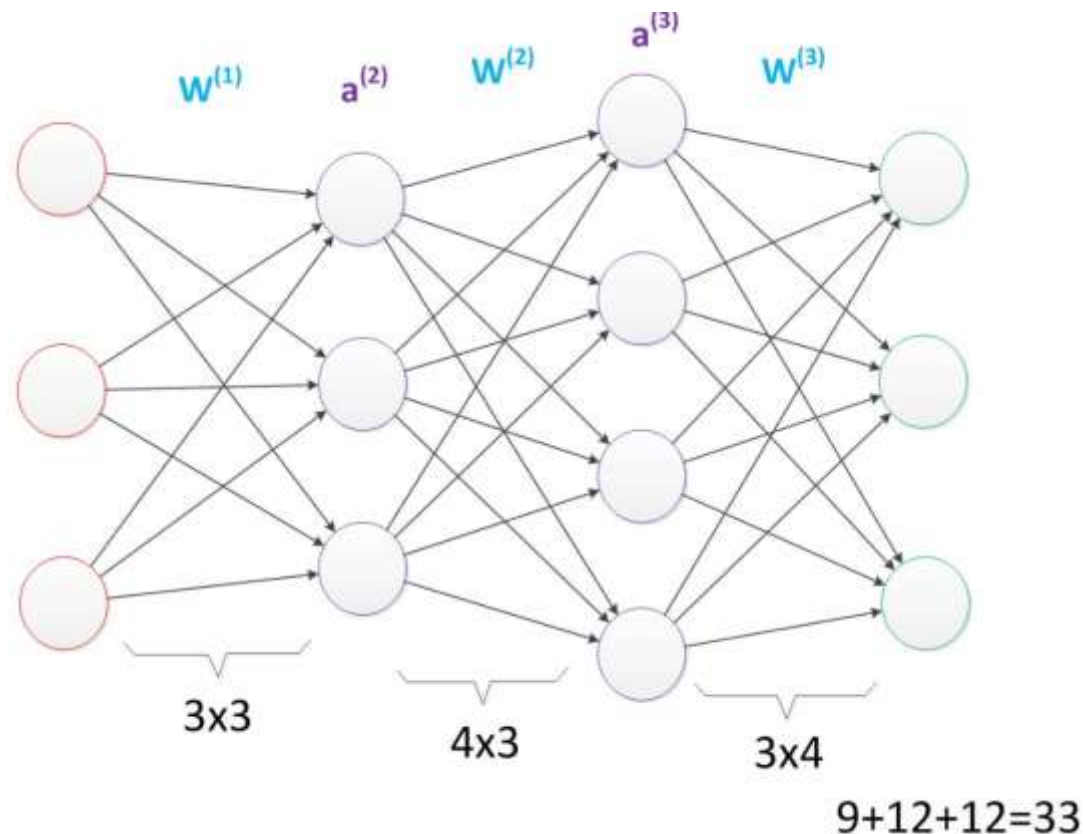
- 很快，深度学习在语音识别领域暂露头角。
- 2012年，深度学习技术又在图像识别领域大展拳脚。Hinton与他的学生在ImageNet竞赛中，用多层的卷积神经网络成功地对包含一千类别的一百万张图片进行了训练，取得了分类错误率15%的好成绩，这个成绩比第二名高了近11个百分点，充分证明了多层神经网络识别效果的优越性。
- 在这之后，关于深度神经网络的研究与应用不断涌现。

□ 多层神经网络—深度学习

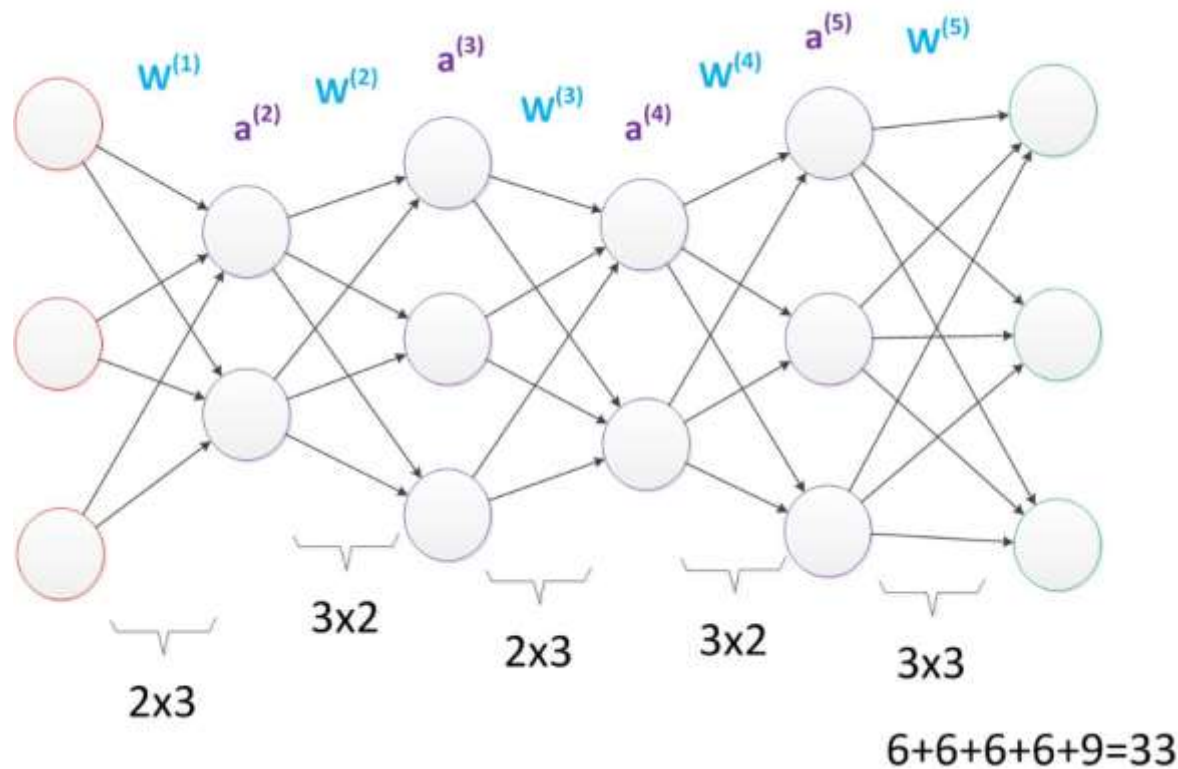
- 在两层神经网络的输出层后面，继续添加层次。原来的输出层变成中间层，新加的层次成为新的输出层。所以可以得到下图。依照这样的方式不断添加，可以得到更多层的多层神经网络。
- 较少参数如图：



- 较多参数如图：
- 虽然层数保持不变，但是第二个神经网络的参数数量却是第一个神经网络的接近两倍之多，从而带来了更好的表示（representation）能力。表示能力是多层神经网络的一个重要性质。



- 更深层次的网络。增加更多的层次有更深入的代表特征, 以及更强的函数模拟能力。
- 随着网络的层数增加, 每一层对于前一层次的抽象表示更深入。目前, 深度神经网络在人工智能界占据统治地位。



第一阶段

- 1943年, McCulloch和Pitts 提出第一个神经元数学模型, 即**M-P模型**, 并从原理上证明了人工神经网络能够计算任何算数和逻辑函数
- 1949年, Hebb 发表《The Organization of Behavior》一书, 提出生物神经元学习的机理, 即**Hebb学习规则**
- 1958年, Rosenblatt 提出**感知机网络** (Perceptron) 模型和其学习规则
- 1960年, Widrow和Hoff提出**自适应线性神经元** (Adaline) 模型和**最小均方学习算法**
- 1969年, Minsky和Papert 发表《Perceptrons》一书, 指出**单层神经网络不能解决非线性问题, 多层网络的训练算法尚无希望**. 这个论断导致神经网络进入低谷

第二阶段

- 1982年, 物理学家Hopfield提出了一种具有联想记忆、优化计算能力的递归网络模型, 即Hopfield 网络
- 1986年, Rumelhart 等编辑的著作《Parallel Distributed Proceesing: Explorations in the Microstructures of Cognition》报告了反向传播算法
- 1987年, IEEE 在美国加州圣地亚哥召开第一届神经网络国际会议 (ICNN)
- 90年代初, 伴随统计学习理论和SVM的兴起, 神经网络由于理论不够清楚, 试错性强, 难以训练, 再次进入低谷

第三阶段

- 2006年, Hinton提出了深度信念网络(DBN), 通过“预训练+微调”使得深度模型的最优化变得相对容易
- 2012年, Hinton 组参加ImageNet 竞赛, 使用 CNN 模型以超过第二名10个百分点的成绩夺得当年竞赛的冠军
- 伴随云计算、大数据时代的到来, 计算能力的大幅提升, 使得深度学习模型在计算机视觉、自然语言处理、语音识别等众多领域都取得了较大的成功 (数据和算力的发展非常重要)

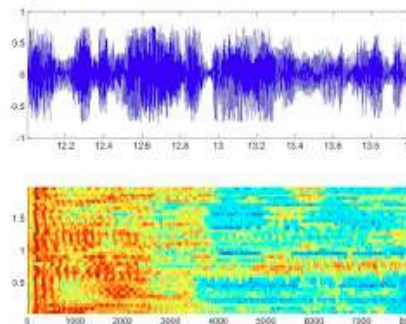
Images & Video



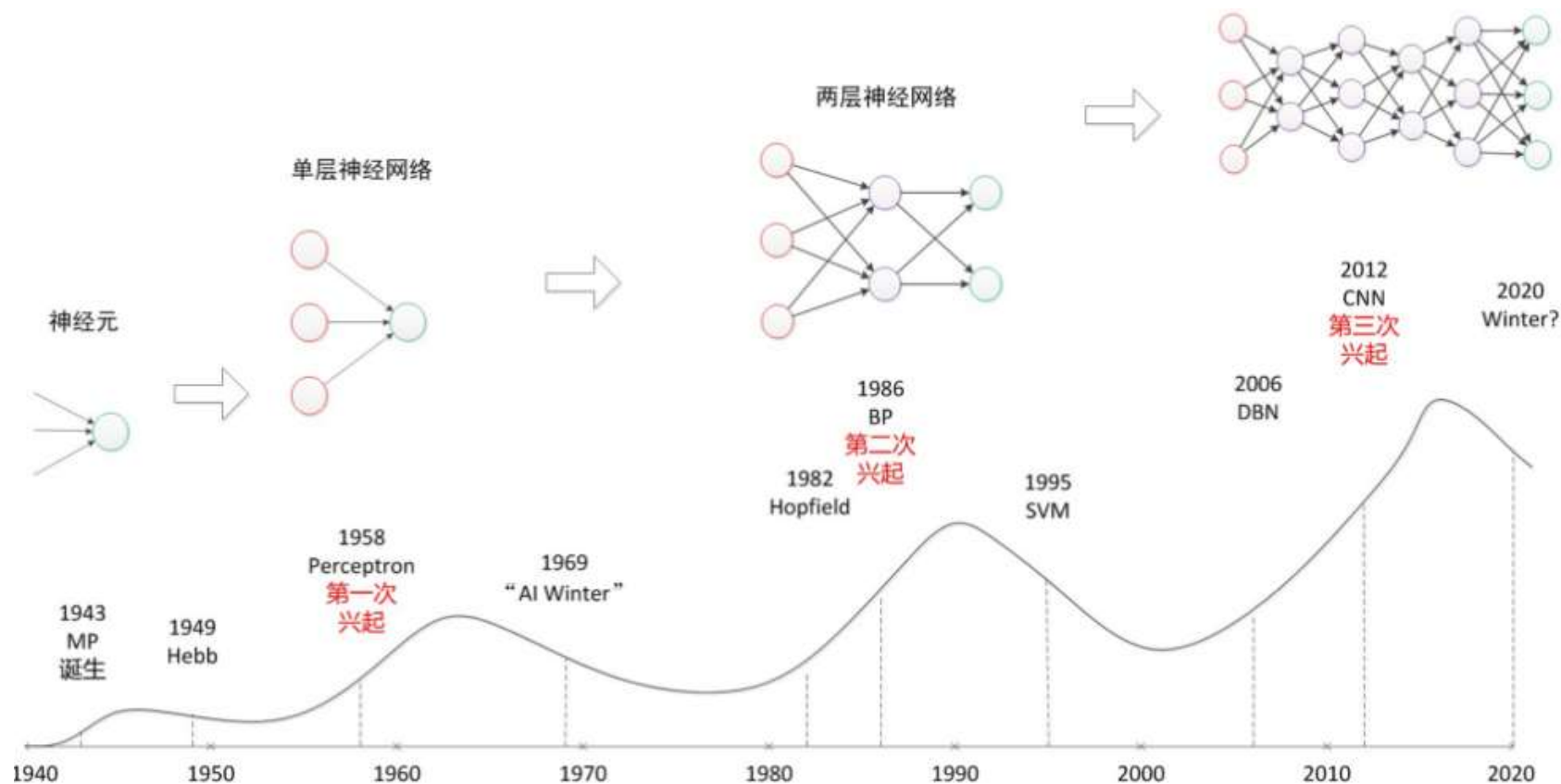
Text & Language



Speech & Audio



三起三落



■ 感知机的负面影响

图灵奖得主，AI之父。创立麻省理工学院人工智能实验室对现代计算机业产生了深远的影响。



- ✓ 感知器只能做简单的线性分类任务。但是当时的人们热情太过于高涨，并没有人清醒的认识到这点。于是，当人工智能领域的巨擘Minsky指出这点时，事态就发生了变化。
- ✓ Minsky(Aug. 9, 1927 – Jan. 24, 2016)在1969年出版了一本叫《Perceptron》的书，里面用详细的数学证明了感知器的弱点，尤其是感知器对XOR（异或）
(playground.tensorflow.org) 这样的简单分类任务都无法解决。而如果将计算层增加到两层，计算量则过大，而且没有有效的学习算法。所以，他认为研究更深层的网络是没有价值的。由于Minsky的巨大影响力以及书中呈现的悲观态度，让很多学者和实验室纷纷放弃了神经网络的研究。神经网络的研究陷入了冰河期。这个时期又被称为“AI winter”。近10年以后，对两层神经网络的研究才带来神经网络的复苏。

第5章 神经网络

5.1 神经元模型

5.2 感知机与多层网络

➡ 5.3 误差逆传播算法

5.4 全局最小与局部极小

5.5 其他常见神经网络

5.6 深度学习

□ 误差逆传播算法

多层网络的学习能力比单层感知机强得多。欲训练多层网络，式(5.1)的简单感知机学习规则显然不够了，需要更强大的学习算法。

- **误差逆传播算法**（ Error Backpropagation，简称BP算法）是一种训练人工神经网络的方法，通过正向传播传递信息、反向传播修正权值实现参数优化，适用于多层前馈神经网络及其他类型网络的训练。该算法由输入层接收信号前向计算输出，根据输出误差逆向逐层计算梯度并更新权重阈值，迭代至满足停止条件。
- **误差逆传播算法**是最成功的训练多层前馈神经网络的学习算法。

□ 误差逆传播算法

- 给定训练集 $D = \{(\mathbf{x}_i, \mathbf{y}_i)\}, \mathbf{x}_i \in R^d, \mathbf{y}_i \in R^l, (i = 1, 2, \dots, m)$, 即输入示例由 d 个属性描述, 输出 l 维实值向量.
- 为方便讨论, 图5.7给出一个拥有 d 个输入神经元, l 个输出神经元, q 个隐层神经元的**多层前向前馈网络结构**.

- 其中记号为:

θ_j : 输出层第 j 个神经元阈值。

γ_h : 隐层第 h 个神经元阈值。

v_{ih} : 输入层第 i 个神经元与隐层第 h 个神经元的连接权重。

w_{hj} : 隐层第 h 个神经元与输出层第 j 个神经元间的连接权重。

$\alpha_h = \sum_{i=1}^d v_{ih} x_i$: 隐层第 h 个神经元接收到的输入。

$\beta_j = \sum_{h=1}^q w_{hj} b_h$: 输出层第 j 个神经元。

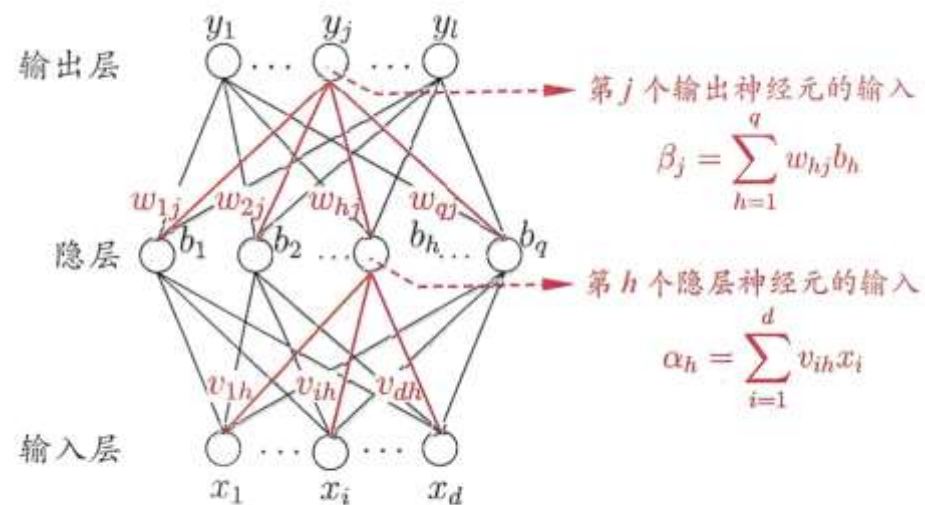


图 5.7 BP 网络及算法中的变量符号

□ 误差逆传播算法

对于训练例 $(\mathbf{x}_k, \mathbf{y}_k)$ ，假设神经网络的实际输出为 $\hat{\mathbf{y}}_k = (\hat{y}_1^k, \hat{y}_2^k, \dots, \hat{y}_l^k)$

■ 前向计算

step1: $b_h = f(\alpha_h - \gamma_h)$ $\alpha_h = \sum_{i=1}^d v_{ih}x_i$

step2: $\hat{y}_j^k = f(\beta_j - \theta_j)$ $\beta_j = \sum_{h=1}^q w_{hj}b_h$

step3: 则网络在 $(\mathbf{x}_k, \mathbf{y}_k)$ 上的均方误差为:

$$E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$$

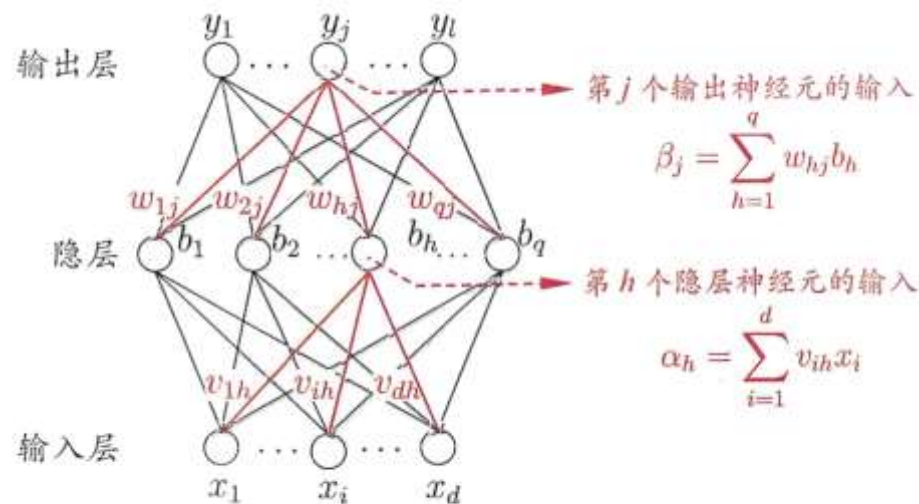


图 5.7 BP 网络及算法中的变量符号

■ 参数数目

权重: v_{ih}, w_{hj} 阈值: θ_j, γ_h ($i = 1, \dots, d, h = 1, \dots, q, j = 1, \dots, l$)

因此网络中需要 $(d + l + 1)q + l$ 个参数需要优化。

■ 参数优化

BP是一个迭代学习算法, 在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计, 任意的参数 的更新估计式为

□ 误差逆传播算法

对于训练例 $(\mathbf{x}_k, \mathbf{y}_k)$ ，假设神经网络的实际输出为 $\hat{\mathbf{y}}_k = (\hat{y}_1^k, \hat{y}_2^k, \dots, \hat{y}_l^k)$

■ 参数数目

网络中有 $(d+l+1)q+l$ 个参数需要优化确定:

- ✓ 输入层到隐层的 $d*q$ 个权值;
- ✓ 隐层到输出层的 $q*l$ 个权值;
- ✓ q 个隐层神经元的阈值;
- ✓ l 个输出层神经元的阈值。

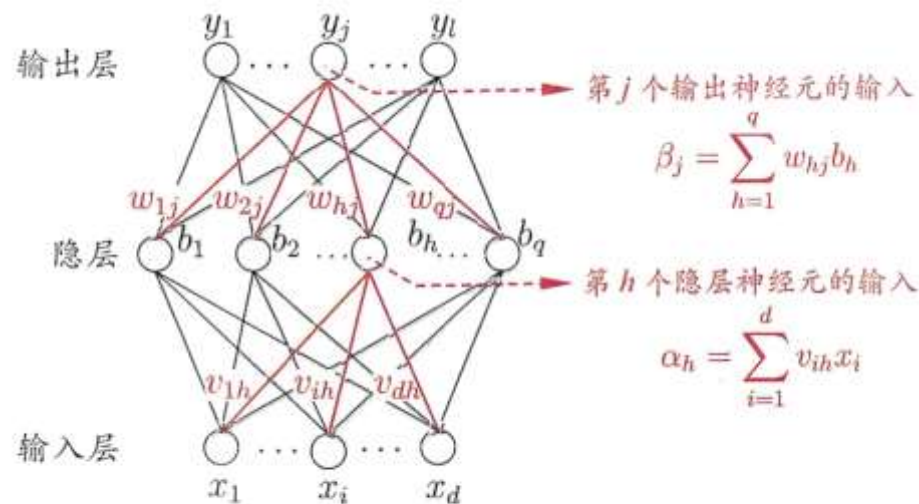


图 5.7 BP 网络及算法中的变量符号

■ 参数优化

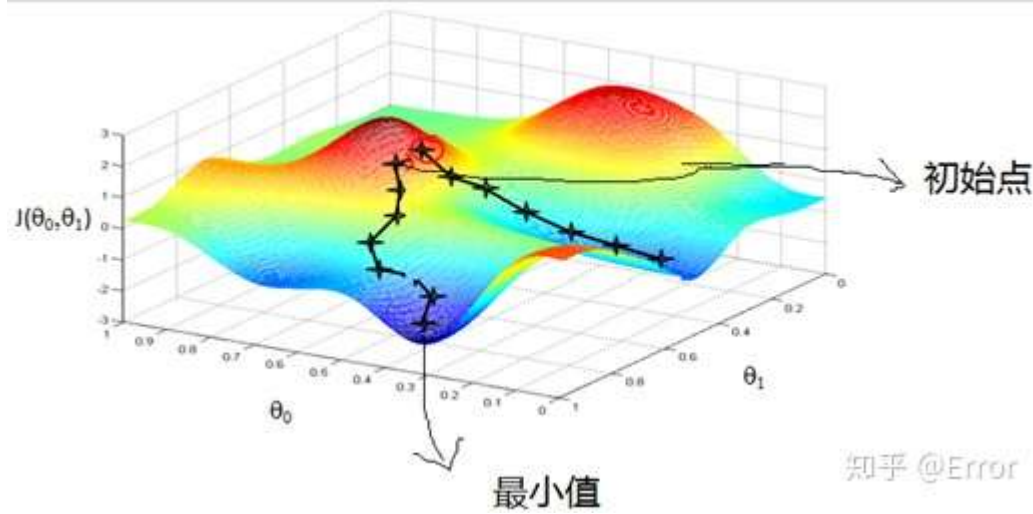
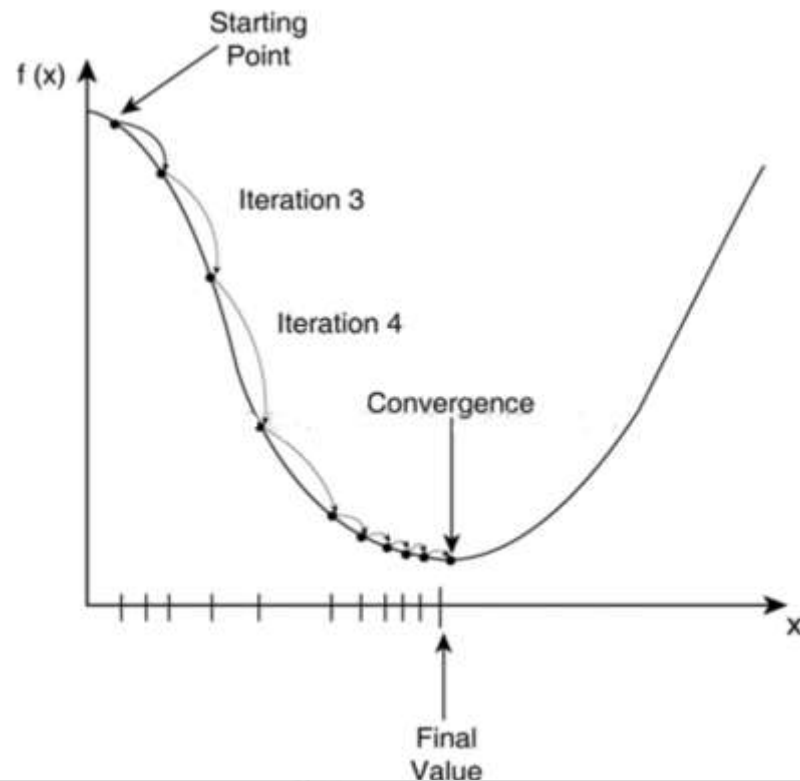
- ✓ BP是一个迭代学习算法, 在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计, 任意的参数 v 的更新估计式为:

$$v \leftarrow v + \Delta v.$$

- ✓ 下面以图5.7中隐层到输出层的连接权 w_{hj} 为例来进行推导。

□ 误差逆传播算法——梯度下降原理

- **梯度**:表示某一函数在该点处的方向导数沿着该方向取得最大值（即最大方向导数的方向），也就是函数在该点处沿着该方向变化最快，变化率最大（为该梯度的模）
- **梯度下降**: 调整参数 θ 使得代价函数 $J(\theta)$ 取得最小值的最基本方法之一。从直观上理解，就是在碗状结构的凸函数上取一个初始值，然后挪动这个值一步步靠近最低点的过程
- 从数学上理解，为了找到最小值点，就应该朝着下降速度最快的方向(导函数/偏导方向)迈进，每次迈进一小步，再看看此时的下降最快方向是哪，再朝着这个方向迈进，直至最低点。



□ 误差逆传播算法——梯度下降原理

■ **例子：**求损失函数 $J(x)=x^2$ 的最小值。利用梯度下降的方法解题步骤如下：

✓ 1、求参数 x 的梯度 $\nabla = \frac{\partial J(x)}{\partial x} = 2x$

✓ 2、向梯度相反方向移动 x 。参数 x 更新为： $x_{n+1}=x_n - \eta \nabla n$

其中： η 为学习率（步长），如果步长足够小，则可以保证每次迭代都在减少，但可能导致收敛太慢；如果步长太大，则不能保证每次迭代都减少，也不能保证收敛。

✓ 3、循环迭代第2步，直到 x 的变化使得 $J(x)$ 在两次迭代间的差值足够小为止。

➤ 假设参数初始值为5，学习率为0.3。

➤ 经过5次迭代后，参数 x 的值变为0.512，这个和参数最优值0已经比较接近了。

使用梯度下降算法优化函数 $J(x) = x^2$

轮数	当前轮参数值	梯度×学习率	更新后参数值
1	5	$2 \times 5 \times 0.3 = 3$	$5 - 3 = 2$
2	2	$2 \times 2 \times 0.3 = 1.2$	$2 - 1.2 = 0.8$
3	0.8	$2 \times 0.8 \times 0.3 = 0.48$	$0.8 - 0.48 = 0.32$
4	0.32	$2 \times 0.32 \times 0.3 = 0.192$	$0.32 - 0.192 = 0.128$
5	0.128	$2 \times 0.128 \times 0.3 = 0.0768$	$0.128 - 0.0768 = 0.0512$

BP 学习算法

- BP算法基于梯度下降策略, 以目标的负梯度方向对参数进行调整. 对误差 E_k , 给定学习率 η 有:

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

- ✓ 注意到 w_{hj} 先影响到第 j 个输出层神经元的输入值 β_j , 再影响到其输出值 $\hat{y}_j^k$, 然后影响到 E_k , 有:

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}$$

- ✓ 根据 β_j 定义, 显然有:

$$\frac{\partial \beta_j}{\partial w_{hj}} = b_h$$

- ✓ Sigmoid函数有一个很好的性质:

$$f'(x) = f(x)(1 - f(x))$$

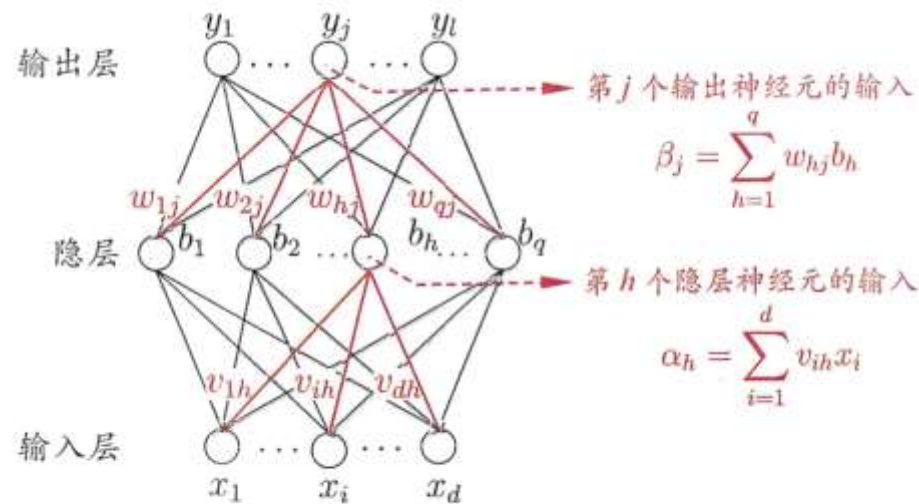


图 5.7 BP 网络及算法中的变量符号

BP 学习算法

✓ 于是，根据上式，有：

$$\begin{aligned} g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k (1 - \hat{y}_j^k) (y_j^k - \hat{y}_j^k) \quad (5.10) \end{aligned}$$

✓ 将式（5.10）和（5.8）代入式（5.7），再代入式（5.6），得到BP算法中关于 w_{hj} 的更新公式：

$$\Delta w_{hj} = \eta b_j g_h \quad (5.11)$$

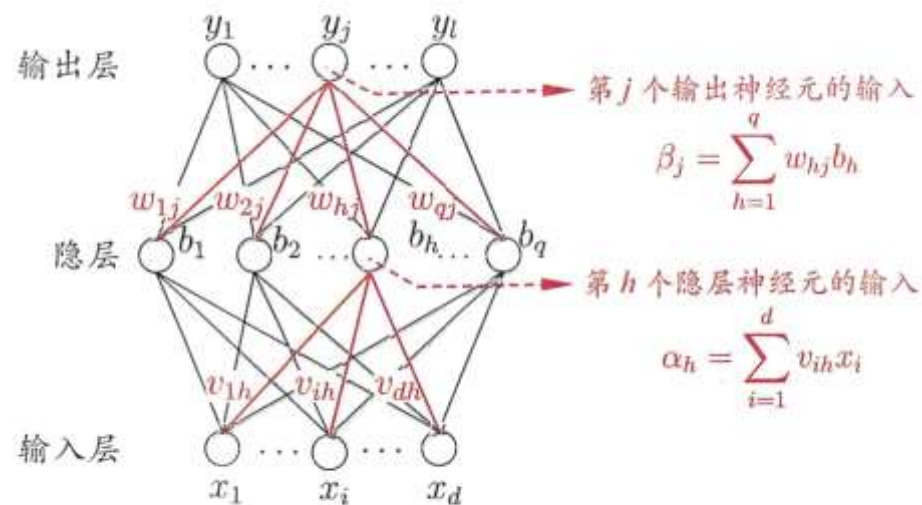


图 5.7 BP 网络及算法中的变量符号

BP 学习算法

- 类似的可以推导出:

$$\Delta \theta_j = -\eta g_j, \quad (5.12)$$

$$\Delta v_{ih} = \eta e_h x_i, \quad (5.13)$$

$$\Delta \gamma_h = -\eta e_h, \quad (5.14)$$

上式其中:

$$\begin{aligned} e_h &= -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} \\ &= -\sum_{j=1}^l \frac{\partial E_k}{\partial \beta_j} \frac{\partial b_h}{\partial \alpha_h} f'(\alpha_h - \gamma_h) \\ &= \sum_{j=1}^h w_{hj} g_j f'(\alpha_h - \gamma_h) \\ &= b_h(1 - b_h) \sum_{j=1}^h w_{hj} g_j. \end{aligned} \quad (5.15)$$

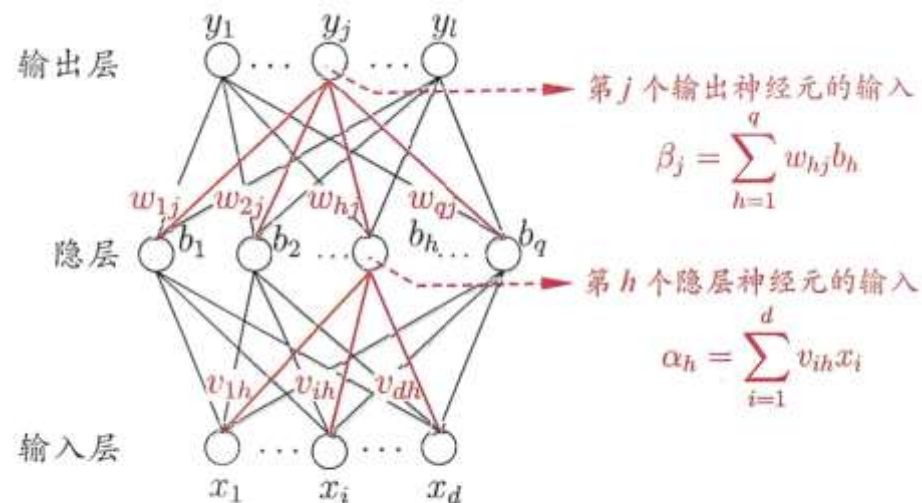


图 5.7 BP 网络及算法中的变量符号

- 学习率 $\eta \in (0, 1)$ 控制着算法每一轮迭代中的更新步长, 若太长则让容易震荡, 太小则收敛速度又会过慢。

■ BP学习算法流程

✓ 给出了BP算法的工作流程。对每个训练样例，BP算法执行以下操作：

- 先将输入示例提供给输入层神经元，然后逐层将信号前传，直到产生输出层的结果；
- 然后计算输出层的误差(第4 - 5 行)，再将误差逆向传播至隐层神经元(第6 行)，
- 最后根据隐层神经元的误差来对连接权和阈值进行调整(第7 行)。
- 该迭代过程循环进行，直到达到某些停止条件为止；例如训练误差已达到一个很小的值。

输入：训练集 $D = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^m$ ；
学习率 η 。

过程：

- 1: 在(0, 1)范围内随机初始化网络中所有连接权和阈值
- 2: **repeat**
- 3: **for all** $(\mathbf{x}_k, \mathbf{y}_k) \in D$ **do**
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\mathbf{y}}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj} , v_{ih} 与阈值 θ_j , γ_h
- 8: **end for**
- 9: **until** 达到停止条件

输出：连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

■ BP学习算法实验

- ✓ 图5.9给出了在2个属性、5个样本的西瓜数据上，随着训练轮数的增加，网络参数和分类边界的变化情况。

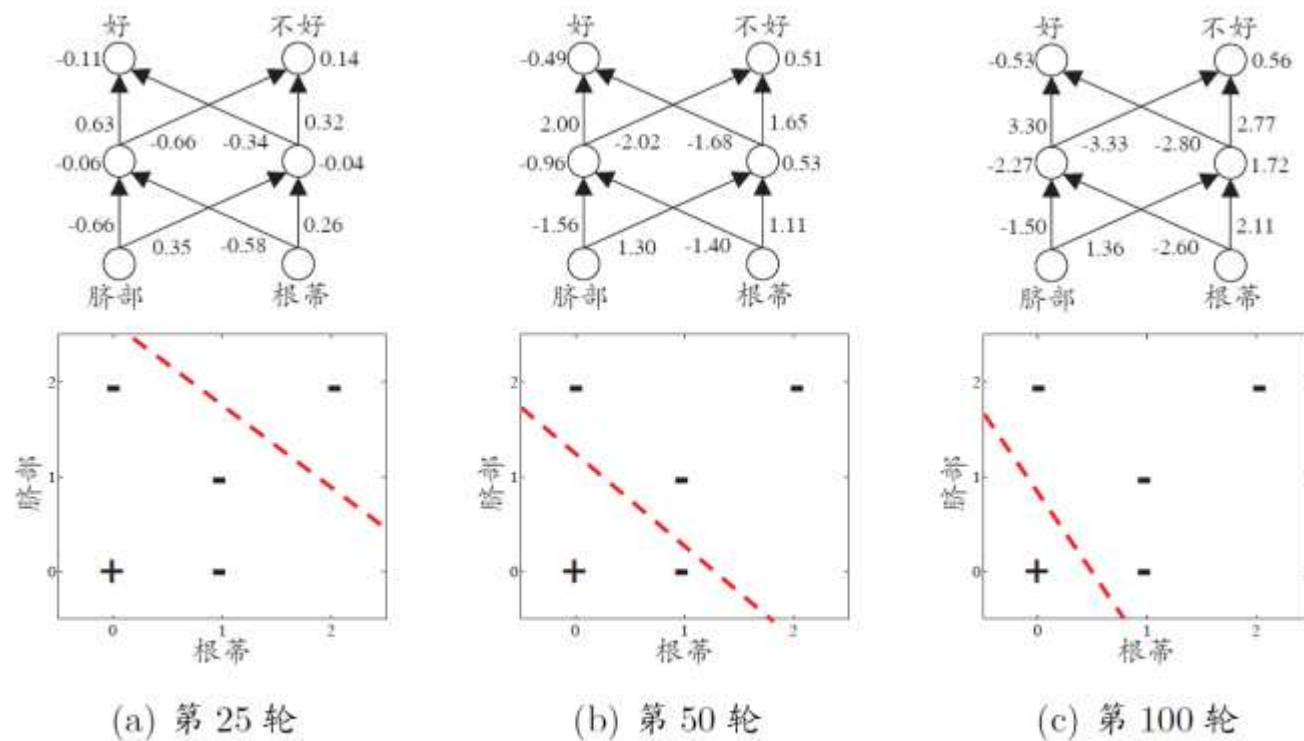


图 5.9 在 2 个属性、5 个样本的西瓜数据上, BP 网络参数更新和分类边界的变化情况

□ 误差逆传播算法——BP学习算法

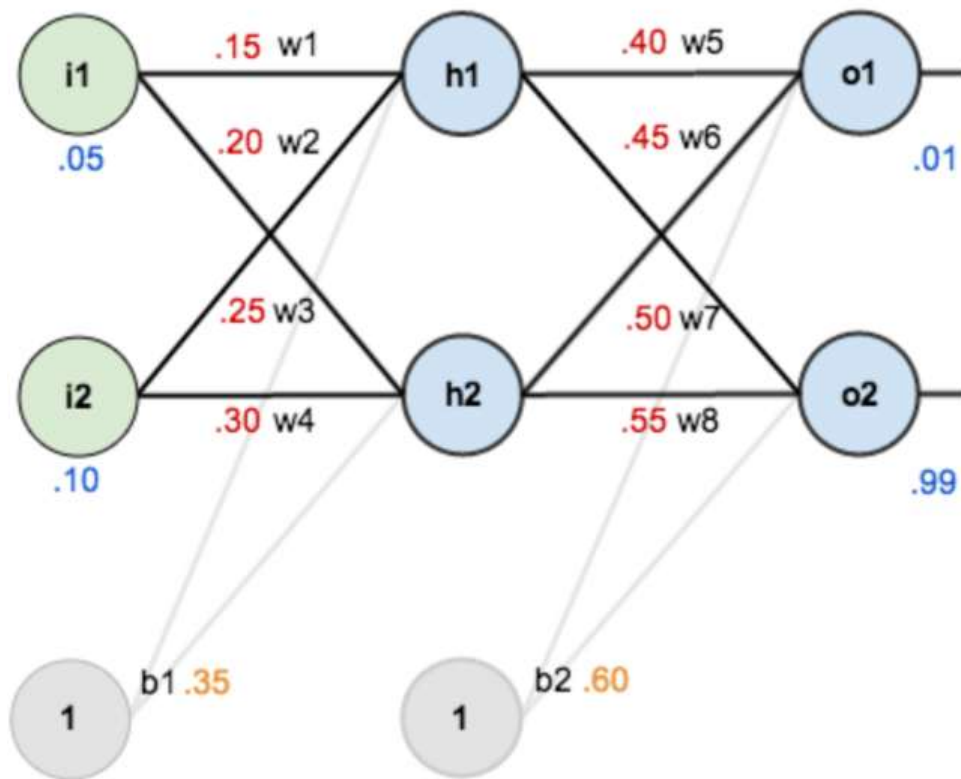
■ BP学习算法实验——实例

输入数据 $i1=0.05$, $i2=0.10$;

输出数据 $o1=0.01$, $o2=0.99$;

初始权重 $w1=0.15, w2=0.20, w3=0.25, w4=0.30$;

$w5=0.40, w6=0.45, w7=0.50, w8=0.55$



目标：给出输入数据 $i1, i2$ (0.05和0.10), 使输出尽可能与原始输出 $o1, o2$ (0.01和0.99)接近。

■ BP学习算法实验——实例

Step 1 前向传播

1.输入层—>隐含层:

计算神经元h1的输入加权和:

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

神经元h1的输出o1:(此处用到激活函数为sigmoid函数):

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

同理, 可计算出神经元h2的输出o2:

$$out_{h2} = 0.596884378$$

□ 误差逆传播算法——BP学习算法

■ BP学习算法实验——实例

2. 隐含层—>输出层:

计算输出层神经元o1和o2的值:

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

$$out_{o2} = 0.772928465$$

- 输出值为[0.75136079, 0.772928465], 与实际值[0.01, 0.99]相差还很远, 现在对误差进行反向传播, 更新权值, 重新计算输出。

■ BP学习算法实验——实例

Step 2 反向传播

1. 计算总误差

总误差: (square error)

$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

但是有两个输出, 所以分别计算o1和o2的误差, 总误差为两者之和:

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

$$E_{o2} = 0.023560026$$

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

□ 误差逆传播算法——BP学习算法

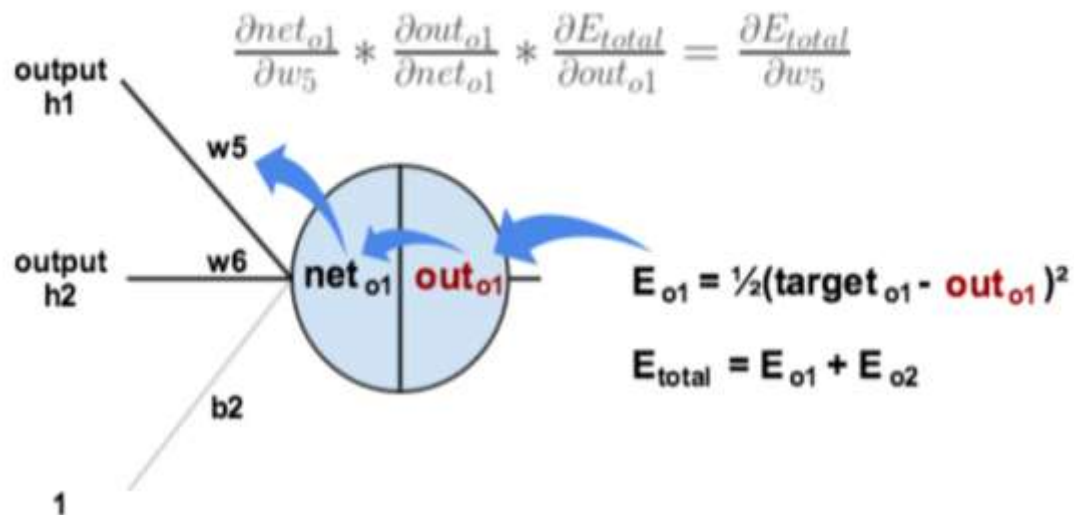
■ BP学习算法实验——实例

2. 隐含层——>输出层的权值更新:

以权重参数 w_5 为例, 如果我们想知道 w_5 对整体误差产生了多少影响, 可以用整体误差对 w_5 求偏导求出: (链式法则)

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

下面的图可以更直观的看清楚误差是怎样反向传播的:



■ BP学习算法实验——实例

现在我们来分别计算每个式子的值：

$$\frac{\partial E_{total}}{\partial out_{o1}}$$

计算：

$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

□ 误差逆传播算法——BP学习算法

■ BP学习算法实验——实例

$$\frac{\partial out_{o1}}{\partial net_{o1}}$$

计算 :

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

■ BP学习算法实验——实例

$$\frac{\partial net_{o1}}{\partial w_5}$$

计算：

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

最后三者相乘：

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

这样我们就计算出整体误差E(total)对w5的偏导值。

■ BP学习算法实验——实例

最后我们来更新w5的值:

$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

同理, 可更新w6,w7,w8:

η

是学习速率, 这里我们取0.5

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

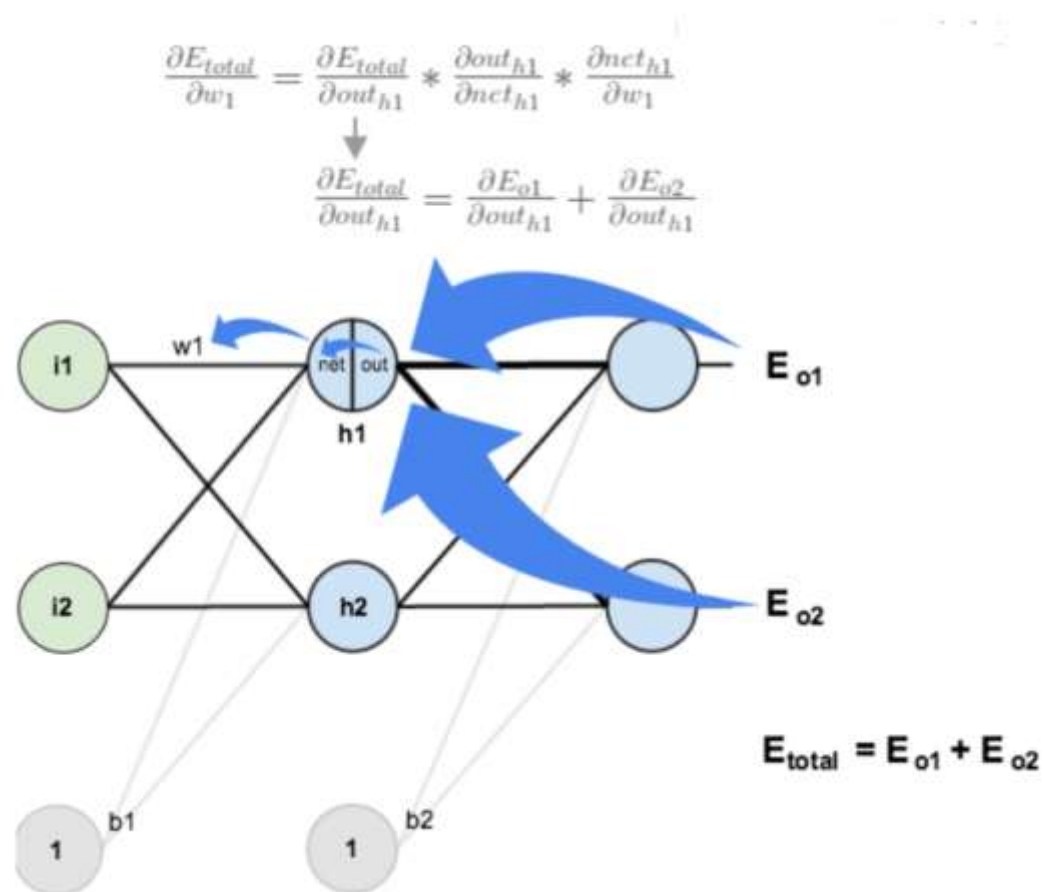
$$w_8^+ = 0.561370121$$

□ 误差逆传播算法——BP学习算法

■ BP学习算法实验——实例

3. 隐含层→隐含层的权值更新

方法其实与上面的差不多。



■ BP学习算法实验——实例

最后，更新 w_1 的权值：

$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

同理， 可更新 w_2, w_3, w_4 的权值：

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

□ 误差逆传播算法——BP学习算法

■ BP学习算法实验—实例

- ✓ 这样误差反向传播法就完成了，最后再把更新的权值重新计算，不停地迭代。
- ✓ 在这个例子中第一次迭代之后，总误差 $E(\text{total})$ 由0.298371109下降至0.291027924。迭代10000次后，总误差为0.000035085，输出为
 $[0.015912196, 0.984065734]$ (原输出为 $[0.01, 0.99]$)，证明效果还是不错的。

□ 误差逆传播算法——BP学习算法

■ 标准BP学习算法

- 每次针对单个训练样例更新权值与阈值.
- 参数更新频繁, 不同样例可能抵消, 需要多次迭代.

■ 累计BP学习算法

- 其优化的目标是最小化整个训练集上的累计误差

$$E = \frac{1}{m} \sum_{k=1}^m E_k$$

- 读取整个训练集一遍才对参数进行更新, 参数更新频率较低.

■ 实际应用

- 但在很多任务中, 累计误差下降到一定程度后, 进一步下降会非常缓慢, 这时标准BP算法往往会获得较好的解, 尤其当训练集非常大时效果更明显.

□ 误差逆传播算法——BP学习算法

■ 多层前馈网络表示能力

只需要一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数（对高维空间的拟合）

[Hornik et al. , 1989]

■ 多层前馈网络局限

- 神经网络由于强大的表示能力，经常遭遇过拟合。表现为：训练误差持续降低，但测试误差却可能上升
- 如何设置隐层神经元的个数仍然是个未决问题。实际应用中通常使用“试错法”调整

■ 缓解过拟合的策略

- 早停：在训练过程中，若训练误差降低，但验证误差升高，则停止训练
- 正则化：在误差目标函数中增加一项描述网络复杂程度的部分，例如连接权值与阈值的平方和

$$E = \lambda \frac{1}{m} \sum_{k=1}^m E_k + (1 - \lambda) \sum_i w_i^2$$

- 使用神经网络训练数据之前，必须确定神经网络层数，以及每层单元个数
- 特征向量在被传入输入层时通常被先标准化（normalize）和0和1之间
- 离散型变量可以被编码成每一个输入单元对应一个特征可能赋的值
- 比如：特征值A可能取三个值（ a_0, a_1, a_2 ），可以使用三个输入单元来代表A；如果 $A=a_0$,那么代表 a_0 的单元值就取1，其他取0; 如果 $A=a_1$,那么代表 a_1 的单元值就取1，其他取0

- 神经网络既可以用来做分类问题，也可以解决回归问题。对于分类问题，如果是2类，可以用一个输出单元表示（0和1分别代表2类），如果多于两类，每一个类别用一个输出单元表示。所以输出层的单元数量通常等于类别的数量。
- 没有明确的规则来设计最好有多少个隐藏层，根据实验测试和误差，以及准确度来实验并改进。
- 初始化权重（weights）和偏置（bias）：随机初始化在-1到1之间，或者-0.5到0.5之间，每个单元有一个偏置。

第5章 神经网络

5.1 神经元模型

5.2 感知机与多层网络

5.3 误差逆传播算法

➡ 5.4 全局最小与局部极小

5.5 其他常见神经网络

5.6 深度学习

□ 全局最小与局部极小

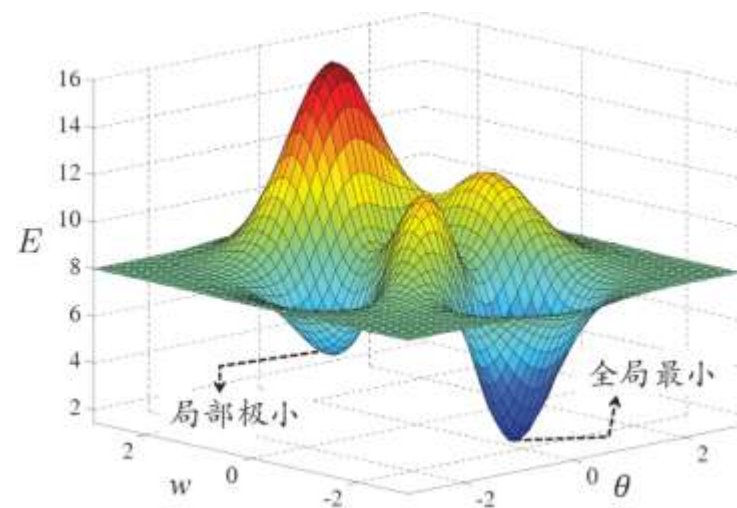
- 若用 E 表示神经网络在训练集上的误差，则它显然是关于连接权 w 和阈值 e 的函数。
- 此时，神经网络的训练过程可看作一个参数寻优过程，即在参数空间中，寻找一组最优参数使得 E 最小。
- 我们常会谈两种“最优”：局部极小和全局最小。
- 对 w^* 和 θ^* ，若存在 $\epsilon > 0$ 使得

$$\forall (w; \theta) \in \{ \|(w; \theta) - (w^*; \theta^*)\| \leq \epsilon \},$$

都有 $E(w; \theta) \geq E(w^*; \theta^*)$ 成立，则 $(w^*; \theta^*)$ 为**局部极小解**；

- 若对参数空间中任意的 $(w; \theta)$ ，都有 $E(w; \theta) \geq E(w^*; \theta^*)$ ，则 $(w^*; \theta^*)$ 为**全局最小解**

- 两者对应的 $E(w^*; \theta^*)$ 分别称为误差函数的局部最小解和全局最小值。



■ “跳出”局部最小的策略

基于梯度的搜索是使用最为广泛的参数寻优方法。如果误差函数仅有一个局部极小，那么此时找到的局部极小就是全局最小；然而，如果误差函数具有多个局部极小，则不能保证找到的解是全局最小。在现实任务中，通常采用以下策略“跳出”局部极小，从而进一步达到全局最小。

- 多组**不同的初始参数**优化神经网络，选取误差最小的解作为最终参数。
- **模拟退火技术 [Aarts and Korst, 1989]**。每一步都以一定的概率接受比当前解更差的结果，从而有助于跳出局部极小。
- **随机梯度下降**。与标准梯度下降法精确计算梯度不同，随机梯度下降法在计算梯度时加入了随机因素。即便陷入局部最小，梯度仍可能不为0。
- **遗传算法 [Goldberg, 1989]**。遗传算法也常用来训练神经网络以更好地逼近全局极小。
- 这些启发式的算法，**理论上缺乏保障**。

第5章 神经网络

5.1 神经元模型

5.2 感知机与多层网络

5.3 误差逆传播算法

5.4 全局最小与局部极小

➡ 5.5 其他常见神经网络

5.6 深度学习

其他常见神经网络

RBF网络 (Radial Basis Function, 径向基函数) [Broomhead and Lowe, 1988]

□ RBF 网络是一种单隐层前馈神经网络, 它使用径向基函数作为隐层神经元激活函数, 而输出层则是隐层神经元输出的线性组合.

□ RBF网络模型

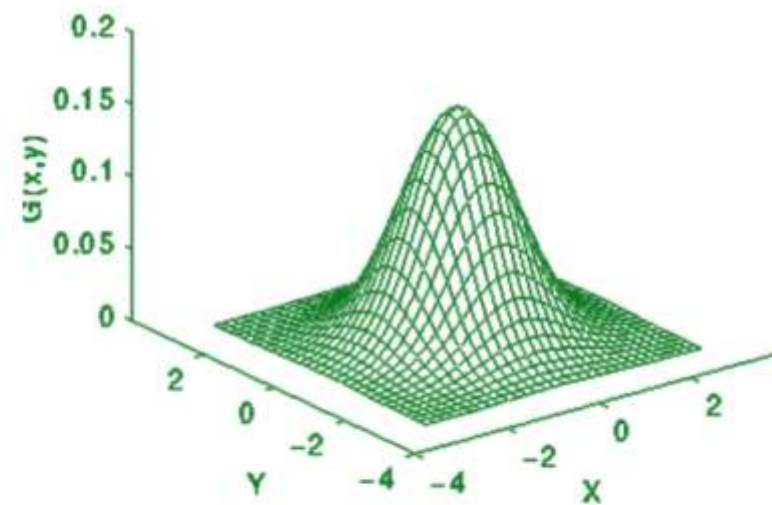
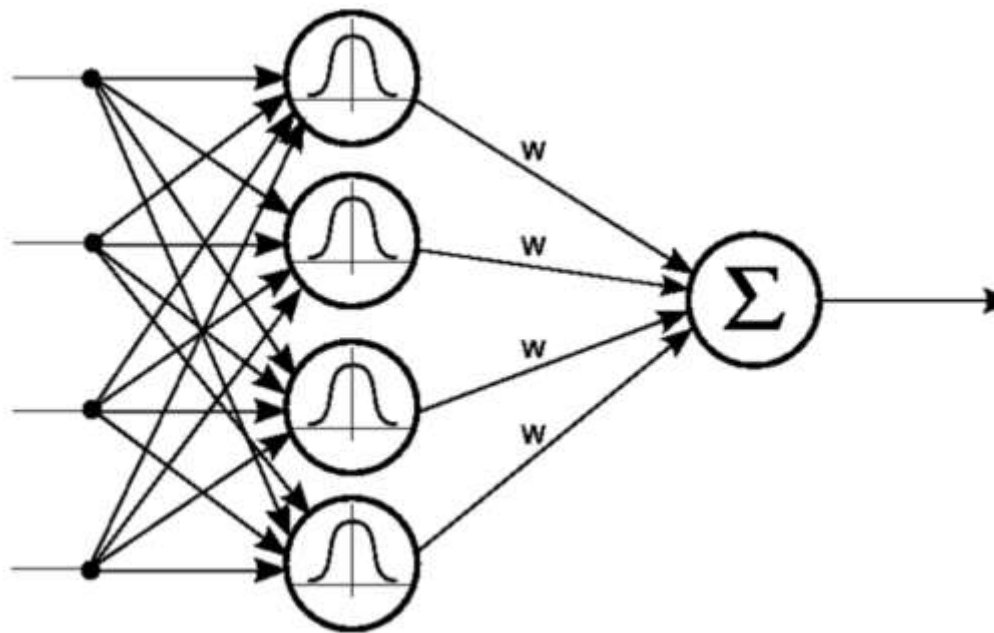
假定输入为 d 维的向量 $\mathbf{x}$, 输出为实值, 则RBF 网络可以表示为

$$\varphi(\mathbf{x}) = \sum_{i=1}^q w_i \rho(\mathbf{x}, \mathbf{c}_i)$$

其中 q 为隐层神经元的个数, $\mathbf{c}_i$ 和 w_i 分别是第 i 神经元对应的中心和权重, $\rho(\mathbf{x}, \mathbf{c}_i)$ 是径向基函数.

常用的高斯径向基函数形如

$$\rho(\mathbf{x}, \mathbf{c}_i) = e^{-\beta_i \|\mathbf{x} - \mathbf{c}_i\|^2}$$



Plot of Radial Basis Function with 2D input

其他常见神经网络

RBF 网络

□ RBF网络性质

具有足够多隐层神经元的RBF 神经网络能以任意精度逼近任意连续函数.

[Park and Sandberg, 1991]

□ RBF网络训练

- Step1:确定**神经元中心**，常用的方式包括随机采样、聚类等
- Step2:利用BP算法等确定参数

□ 特点:

- 只有三层，BP可以多层
- 局部逼近（仅跟局部样本有关），BP网络是全局逼近（跟所有样本有关）
- 训练速度快

其他常见神经网络

ART (Adaptive Resonance Theory,自适应谐振理论) 网络

□ 竞争学习

竞争学习是神经网络中一种常用的无监督学习策略, 在使用该策略时, 网络的输出神经元相互竞争, 每一时刻仅有一个神经元被激活, 其他神经元的状态被抑制。

□ ART 网络 [Carpenter and Grossberg, 1987]

- ART 网络是竞争学习的重要代表
- ART 网络由比较层、识别层、识别阈值和重置模块构成
- 比较层负责接收输入样本, 并将其传送给识别层神经元
- 识别层每个神经元对应一个模式类, 神经元的数目可在训练过程中动态增长以增加新的模式类

其他常见神经网络

ART 网络

□ ART 网络性能依赖于识别阈值

- 识别阈值高时, 输入样本将会分成比较多、得到较精细分类
- 识别阈值低时, 输入样本将会分成比较少、产生较粗略分类

□ ART 网络的优势

- ART较好的解决了竞争学习中的“可塑性-稳定性窘境”, 可塑性是指神经网络要有学习新知识的能力; 稳定性是指神经网络在学习新知识时要保持对旧知识的记忆.
- **ART 网络可以增量学习或在线学习**

□ ART 网络的发展

ART2 网络、FuzzyART 网络、ARTMAP 网络

其他常见神经网络

SOM (Self-Organizing Map ,自组织映射) 网络 [Kohonen, 1982]

- SOM 网络是一种竞争型的无监督神经网络，它可将高维数据映射到低维空间（通常为2维），同时保持输入数据在高维空间的拓扑结构，即将高维空间中相似的样本点映射到网络输出层中邻近神经元。
- 如图，SOM 网络中的输出层神经元以矩阵方式排列在二维空间中，每个神经元都拥有一个权值向量，网络在接收输入向量后，将会确定输出层获胜神经元，它决定了该输入向量在低维空间中的位置。

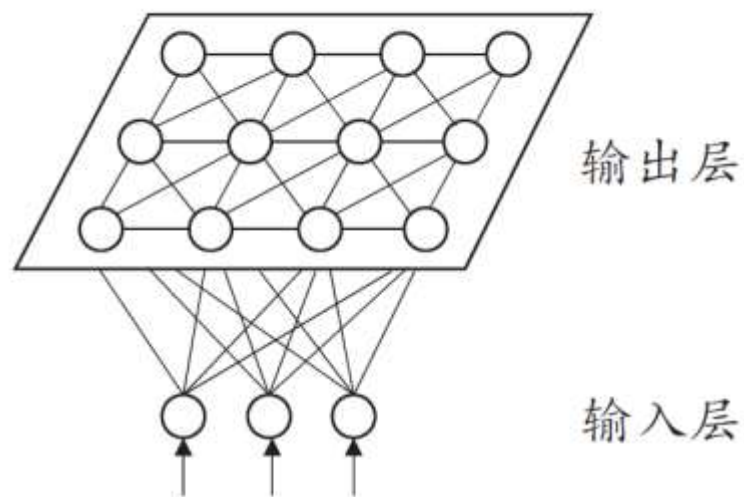


图 5.11 SOM 网络结构

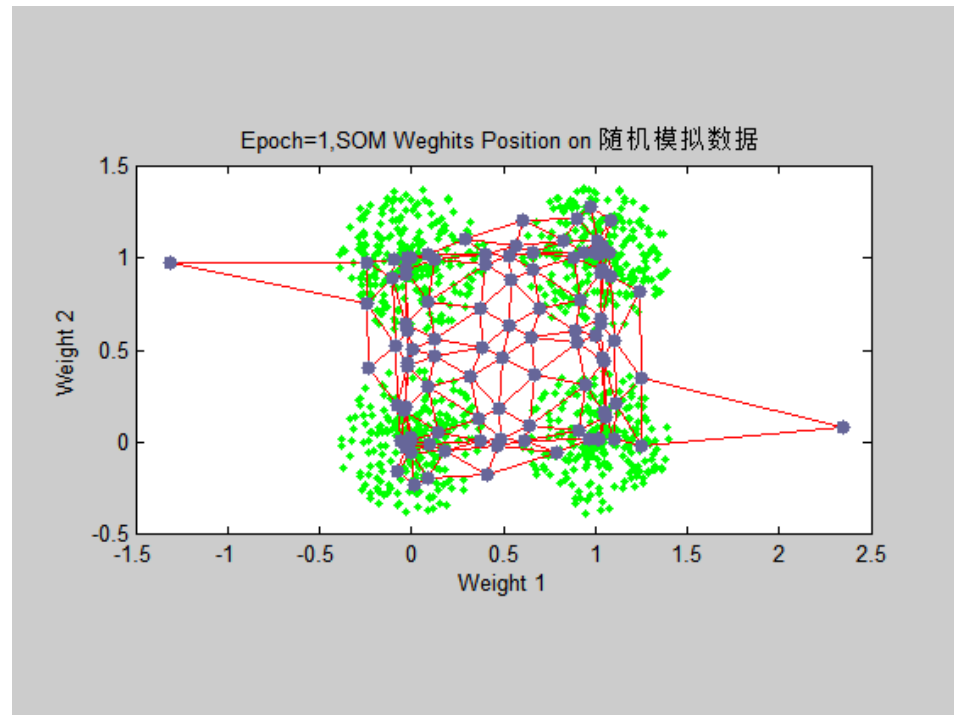
其他常见神经网络

SOM 网络 [Kohonen, 1982]

□ SOM 网络训练

Step1:接受到一个训练样本后, 每个输出层神经元计算该样本与自身携带的权向量之间的距离, 距离最近的神经元成为竞争获胜者

Step2:最佳匹配单元及其近邻神经元的权值将被调整, 使得这些权向量与当前输入样本的距离缩小



其他常见神经网络

级联相关网络 [Fahlman and Lebiere 1990]

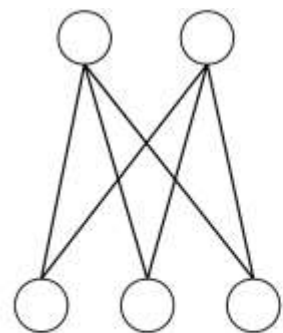
级联相关网络不仅利用训练样本优化连接权值, 阈值参数, **将网络的结构也当做学习的目标之一**, 希望在训练过程中找到适合数据的网络结构.

□ 级联与相关

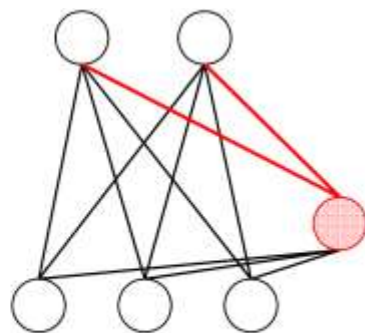
级联: 建立层次连接的层级结构

相关: 最大化神经元的输出与网络误差之间的相关性来训练相关参数

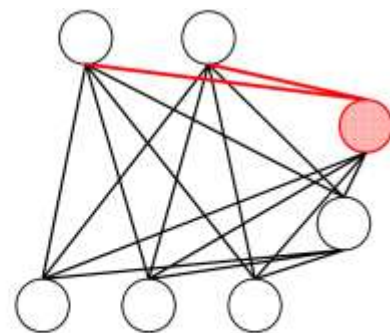
□ 网络优化演示



(a) 初始状态



(b) 增加一个隐层结点



(c) 增加第二个隐层结点

级联相关网络的训练过程。新的隐结点加入时, 红色连接权通过最大化新结点的输出与网络误差之间的相关性来进行训练。

其他常见神经网络

Elman 网络 [Elman 1990]

□ 递归神经网络RNN

- 允许网络中出现环形结构, 使得神经元的输出反馈回来作为输入信号
- t 时刻网络的输出状态: 由 t 时刻的输入状态和 $t-1$ 时刻的网络状态决定

□ Elman 网络

Elamn 网络是常用的递归神经网络之一, 结构如图所示, 这种结构与前馈神经网络很相似, 但是隐层神经元的输出被反馈回来, 与下一时刻输入层神经元提供的信号一起, 作为隐层神经元在下一时刻的输入

□ 训练算法

推广的BP算法. [Pineda, 1987]

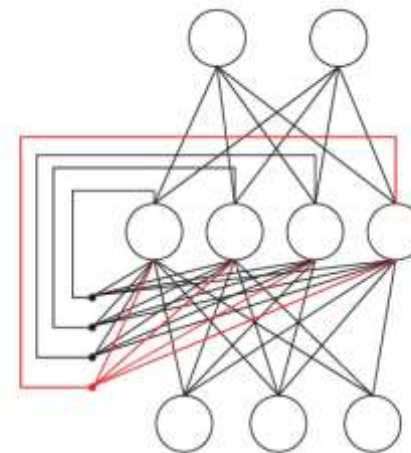


图 5.13 Elman 网络结构

其他常见神经网络

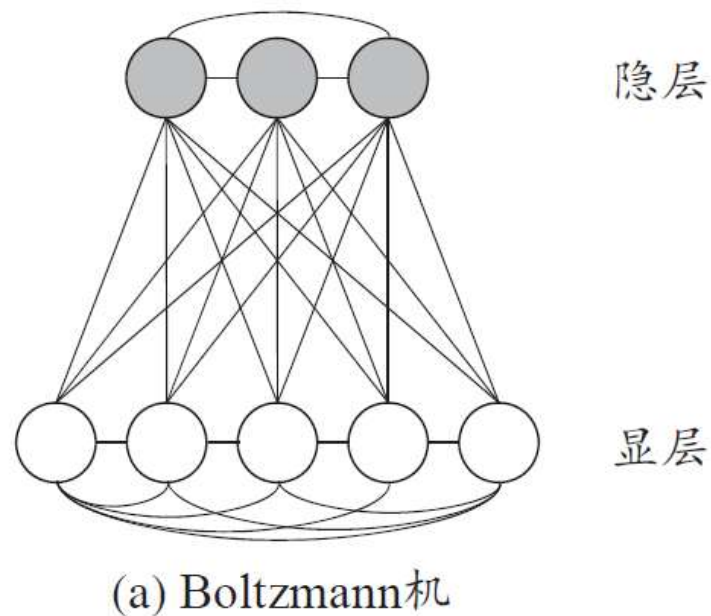
Boltzmann 机

□ 能量模型

神经网络中有一类模型为网络定义一个“能量”，能量最小化时网络达到理想状态，而网络的训练就是在最小化这个能量函数。

□ Boltzmann 机

- Boltzmann 机就是一种基于能量的模型
- 结构：显层与隐层
 - 显层：数据的输入输出
 - 隐层：数据的内在表达
- 神经元
 - 布尔型，即只能取0和1 两种状态，其中1 表示激活，0表示抑制。



其他常见神经网络

Boltzmann 机 [Ackley et al. , 1985]

□ Boltzmann 机能量

令状态向量 $\mathbf{s} \in \{0, 1\}^n$ 表示 n 个神经元的状态，则其对应的 Boltzmann 机能量定义为

$$E(\mathbf{s}) = - \sum_{i=1}^{n-1} \sum_{j=i+1}^n w_{ij} s_i s_j - \sum_{i=1}^n \theta_i s_i$$

其中 w_{ij} 表示两个神经元之间的连接权值， θ_i 表示神经元的阈值。

□ Boltzmann 分布

网络中的神经元以任意不依赖与输入值得顺序进行更新，则网络最终将达到 Boltzmann 分布，此时状态向量出现的概率将仅由其能量与所有可能状态向量的能量确定：

$$P(\mathbf{s}) = \frac{e^{-E(\mathbf{s})}}{\sum_t e^{-E(t)}}$$

其他常见神经网络

Boltzmann 机 [Ackley et al. , 1985]

□ Boltzmann 机训练

- 将每个训练样本视为一个状态向量，使其出现的概率尽可能大
- 标准的 Boltzmann 机是一个全连接图，训练网络的复杂度很高，这使其难以用于解决现实任务
- 现实中常用受限 Boltzmann 机，简称RBM。RBM 仅保留显层与隐层之间的连接，从而将Boltzmann机结构由完全图简化为二部图
- 受限：层内之间没有通信

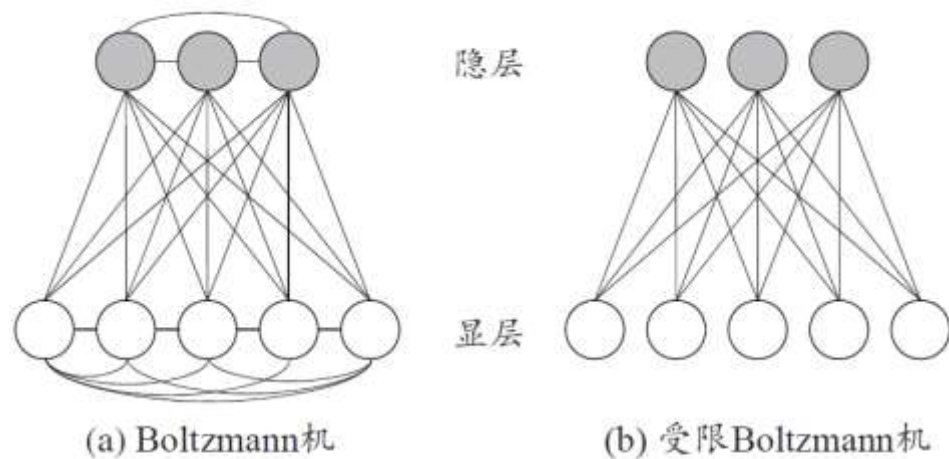


图 5.14 Boltzmann 机与受限 Boltzmann 机

其他常见神经网络

受限 Boltzmann 机 [Ackley et al. , 1985]

□ 受限 Boltzmann 机常用“对比散度”（简称:CD）算法 [Hinton, 2010] 来进行训练

- 假定网络中有 d 个显层神经元 q 个隐层神经元，令 $\mathbf{v}$ 和 $\mathbf{h}$ 分别是显层与隐层的状态向量，由于同一层内不存在连接，有

$$P(\mathbf{v}|\mathbf{h}) = - \prod_{i=1}^d P(v_i|\mathbf{h}),$$

$$P(\mathbf{h}|\mathbf{v}) = - \prod_{i=1}^q P(h_i|\mathbf{v}),$$

- CD算法对每个训练样本 $\mathbf{v}$ 先计算出隐层神经元状态的概率分布，然后根据这个概率分布采样得到 $\mathbf{h}$ ；类似的方法从 $\mathbf{h}$ 中产生 $\mathbf{v}'$ ，再从 $\mathbf{v}'$ 中产生 $\mathbf{h}'$ ；连接权重的更新公式为：

$$\Delta w = \mathbf{v}\mathbf{h}^T - \mathbf{v}'\mathbf{h}'^T$$

第5章 神经网络

5.1 神经元模型

5.2 感知机与多层网络

5.3 误差逆传播算法

5.4 全局最小与局部极小

5.5 其他常见神经网络

 5.6 深度学习

深度学习

深度学习模型

- 典型的深度学习模型就是很深层的神经网络.

□ 模型复杂度

- 增加隐层神经元的数目（模型宽度）
- 增加隐层数目（模型深度）
- 从增加模型复杂度的角度看，增加隐层的数目比增加隐层神经元的数目更有效。这是因为增加隐层数不仅增加了拥有激活函数的神经元数目，还增加了激活函数嵌套的层数。

□ 复杂模型难点

多隐层网络难以直接用经典算法（例如标准BP算法）进行训练，因为误差在多隐层内逆传播时，往往会“发散”而不能收敛到稳定状态。

深度学习

复杂模型训练方法

□ 预训练+微调

- **预训练**: 监督逐层训练是多隐层网络训练的有效手段, 每次训练一层隐层结点, 训练时将上一层隐层结点的输出作为输入, 而本层隐结点的输出作为下一层隐结点的输入, 这称为“预训练”.
- **微调**: 在预训练全部完成后, 再对整个网络进行微调训练. 微调一般使用**BP**算法.

□ 例子: 深度信念网络 [Hinton et al. , 2006]

- 结构: 每一层都是一个受限 Boltzmann 机
- 训练方法: 无监督预训练+BP 微调

□ 分析

预训练+微调 的做法可以**视为将大量参数分组**, 对每组先找到局部看起来比较好的设置, 然后再基于这些**局部较优的结果联合起来进行全局寻优**.

深度学习

复杂模型训练方法

□ 权共享

- 一组神经元使用相同的连接权值.
- **权共享策略**在卷积神经网络(CNN)[LeCun and Bengio, 1995; LeCun et al. , 1998]中发挥了重要作用.

□ 卷积神经网络

结构: CNN复合多个卷积层和采样层对输入信号进行加工, 然后在连接层实现与输出目标之间的映射.

深度学习

□ 卷积神经网络

- **卷积层**：每个**卷积层**包含多个特征映射，每个特征映射是一个由多个神经元构成的“平面”，通过一种卷积滤波器提取的一种特征
- **采样层**：亦称“汇合层”，其作用是基于局部相关性原理进行亚采样，从而在减少数据量的同时保留有用信息
- **连接层**：每个神经元被全连接到上一层每个神经元，本质就是传统的神经网络，其目的是通过连接层和输出层的连接完成识别任务

□ 卷积神经网络激活函数

在CNN中通常将 sigmoid 激活函数替换为修正的**线性函数Relu**

$$f(x) = \max(0, x)$$

□ 卷积神经网络训练

CNN 可以用**BP**进行训练，但在训练中，无论是卷积层还是采样层，每一组神经元都是用相同的连接权，从而大幅减少了需要训练的参数数目

深度学习

□ 以CNN进行手写数字识别任务为例

- ✓ 网络输入是一个 32×32 的手写数字图像，输出是其识别结果。
- ✓ CNN复合多个“卷积层”和“采样层”对输入信号进行加工，然后在连接层实现与输出目标之间的映射。

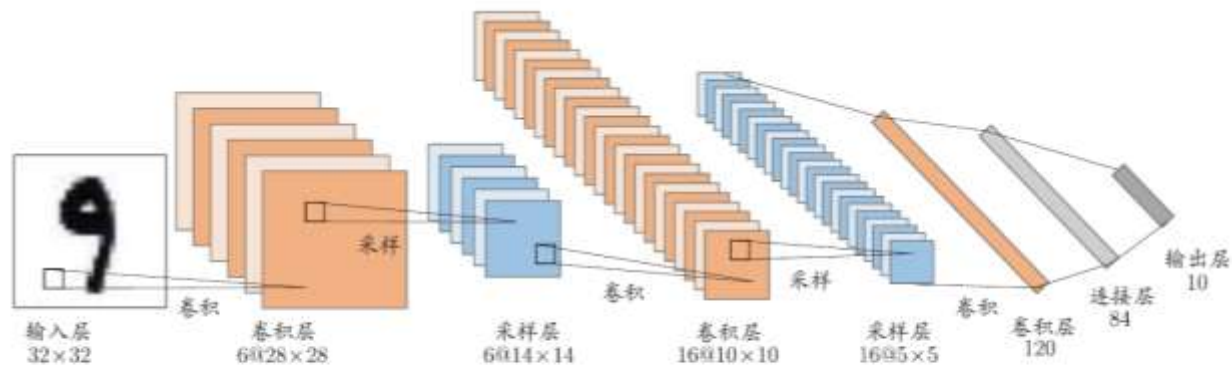
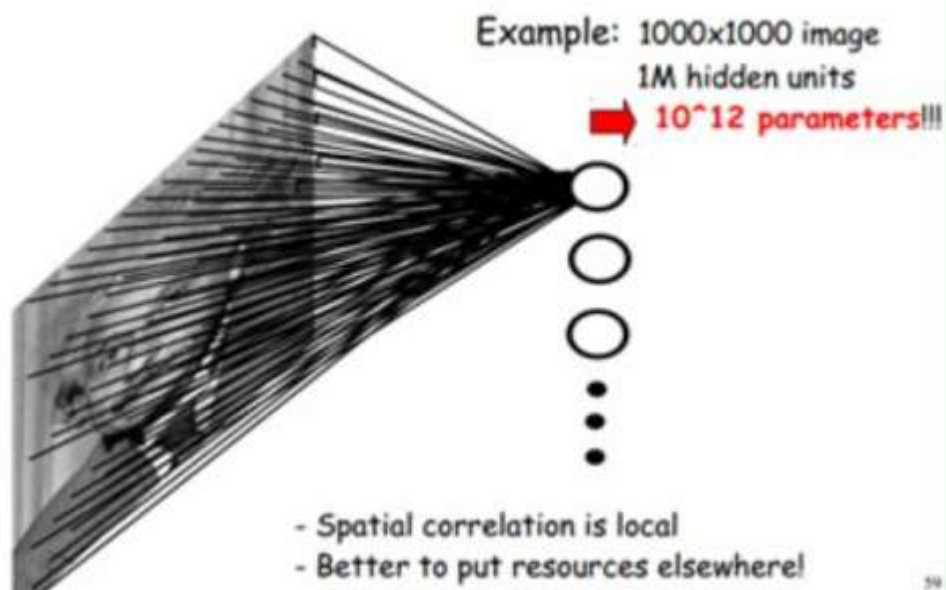


图 5.15 卷积神经网络用于手写数字识别 [LeCun et al., 1998]

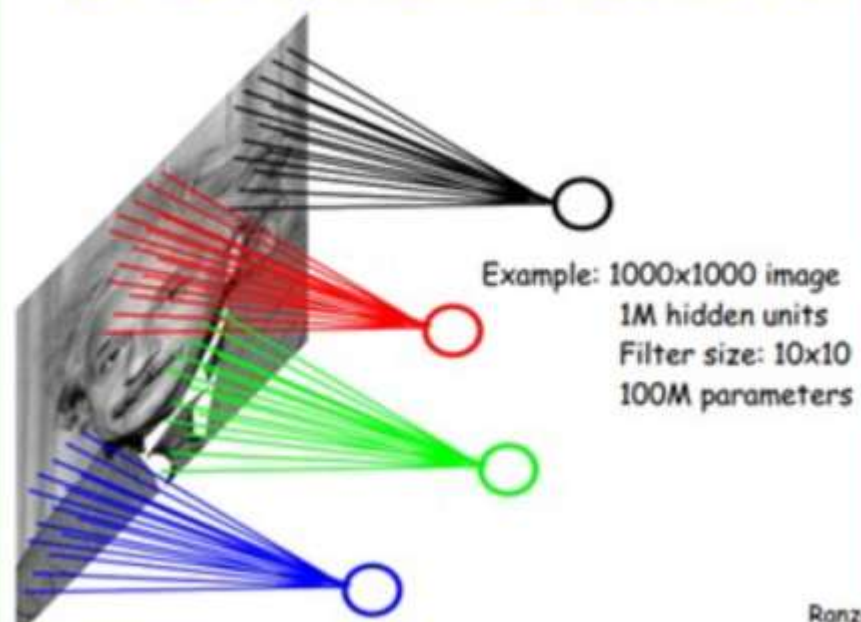
- ✓ 每个卷积层都包含多个特征映射(feature map)，每个特征映射是一个由多个神经元构成的“平面”，通过一种卷积滤波器提取输入的一种特征。例如，上图中第一个卷积层由6个特征映射构成，每个特征映射是一个 28×28 的神经元阵列，其中每个神经元负责从 5×5 的区域通过卷积滤波器提取局部特征。
- ✓ 采样层亦称为“汇合”(pooling)层，其作用是基于局部相关性原理进行亚采样，从而在减少数据量的同时保留有用信息。上图中第一个采样层有6个 14×14 的特征映射，其中每个神经元与上一层中对应特征映射的 2×2 领域相连，并据此计算输出。
- ✓ 通过复合卷积层和采样层，上图中的CNN将原始图像映射成120维特征向量，最后通过一个由84个神经元构成的连接层和输出层连接完成识别任务。
- ✓ CNN可用BP算法进行训练，但在训练中，无论是卷积层还是采样层，其每一组神经元(图中的每个“平面”)都是用相同的连接权，从而大幅减少了需要训练的参数数目。

FULLY CONNECTED NEURAL NET



59

LOCALLY CONNECTED NEURAL NET



卷积

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

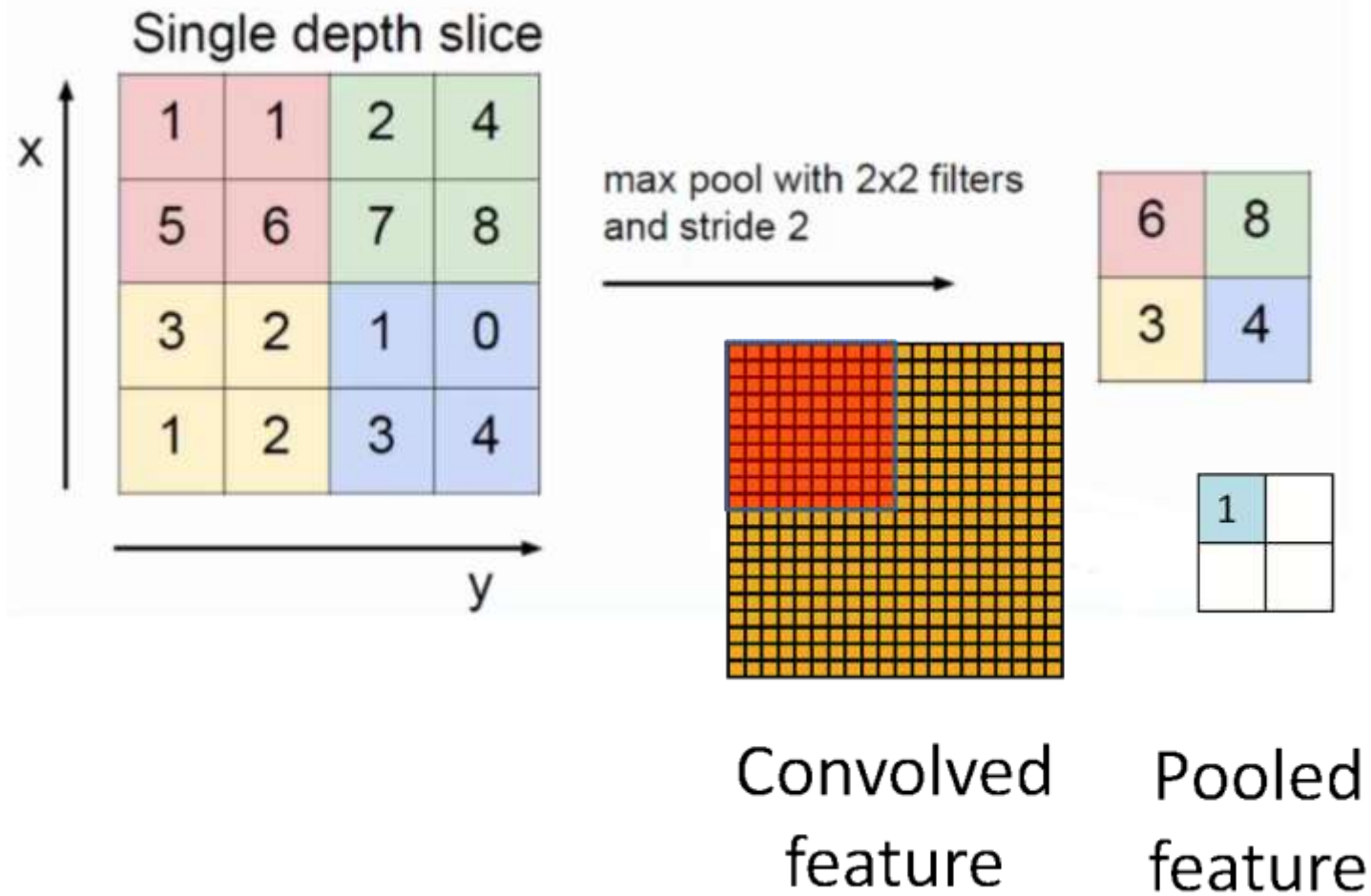
Image

4		

Convolved
Feature

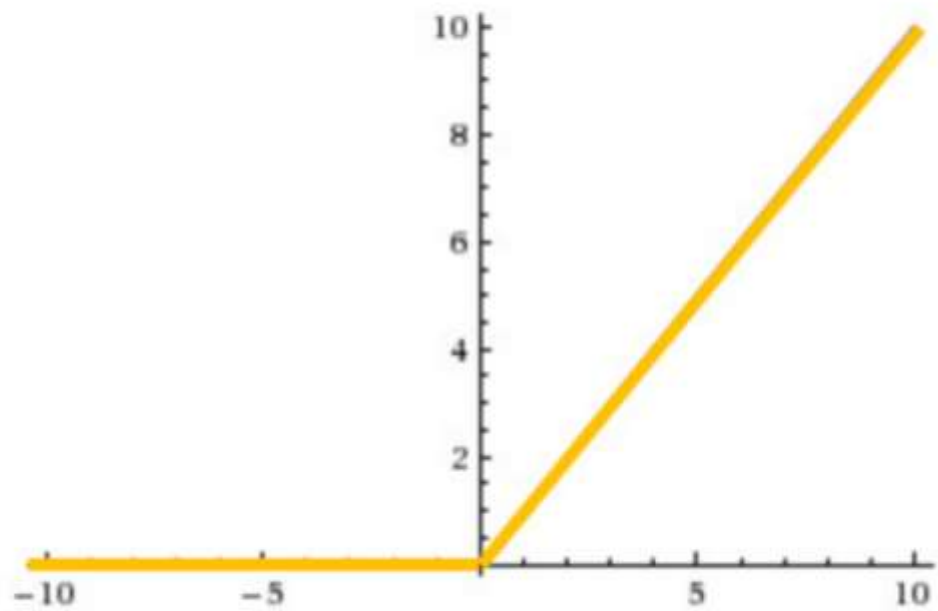
池化

MAX POOLING



激活函数

ReLU



$$F(x) = \max(0, x)$$

Softmax层

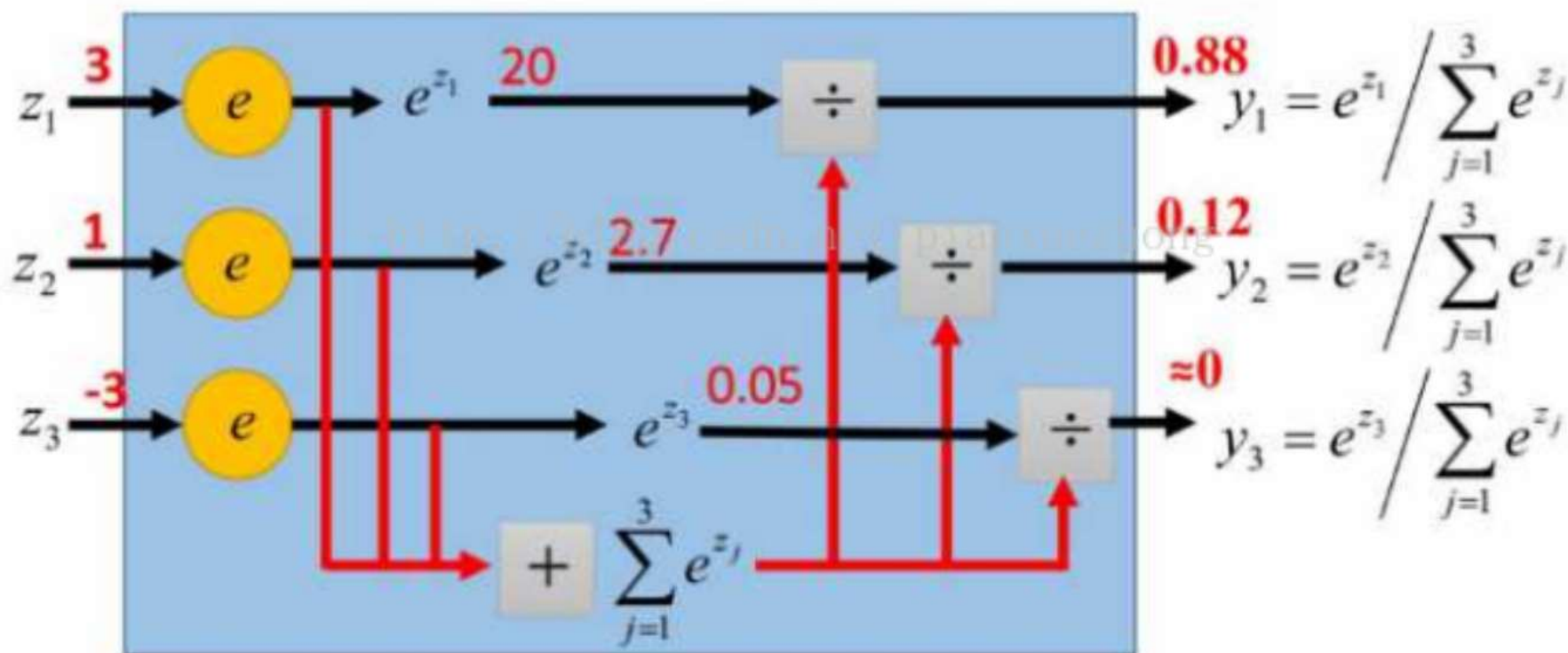
- Softmax layer as the output layer

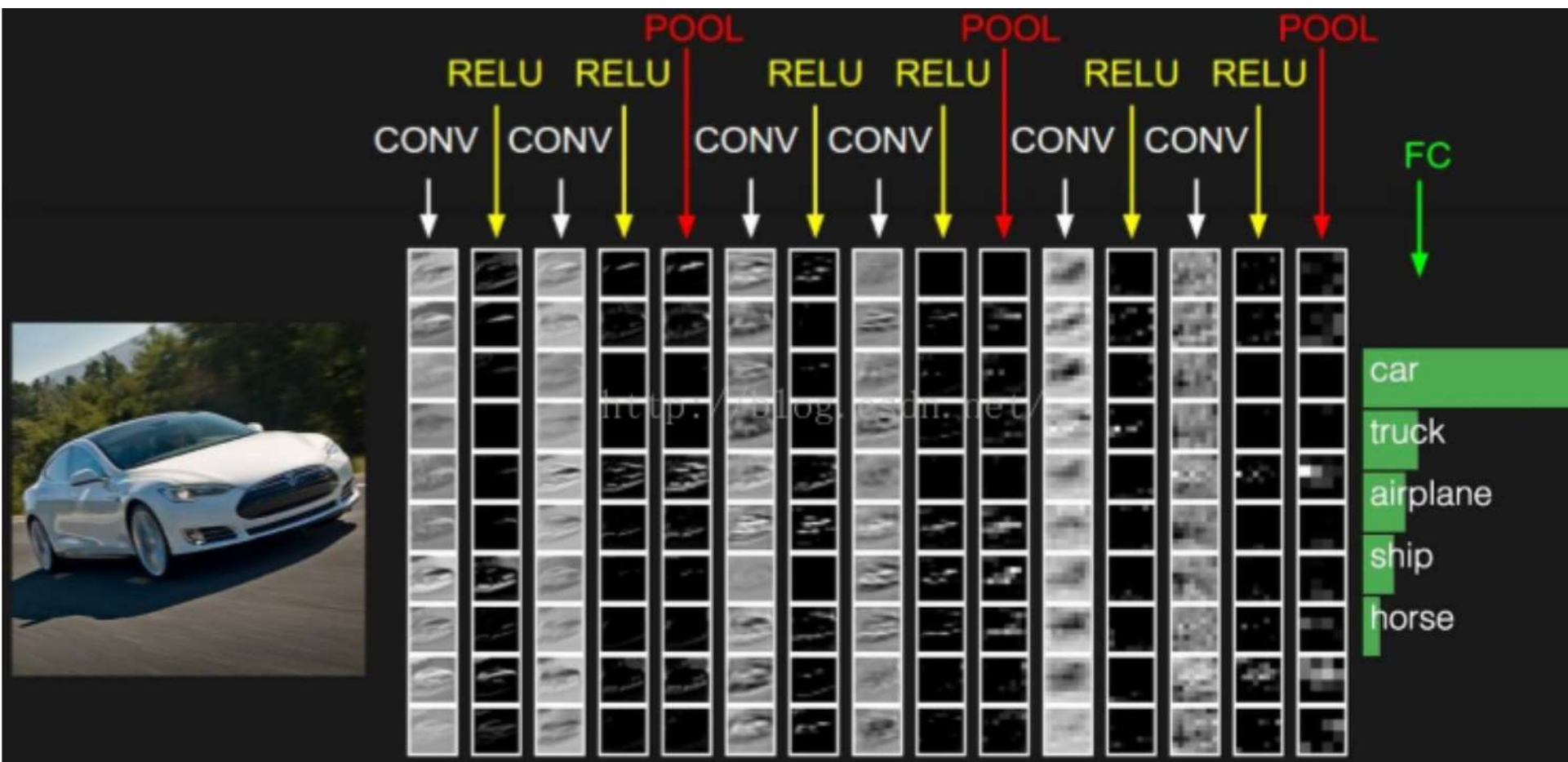
Probability:

■ $1 > y_i > 0$

■ $\sum_i y_i = 1$

Softmax Layer





深度学习

□ 主要卷积神经网络模型

AlexNet:

(2012 ILSVRC top 5 test error rate of 15.4%)

VGGnet:

(ILSVRC 2014 7.3% error rate)

GoogleNet:

(ILSVRC 2014 top 5 error rate of 6.7%)

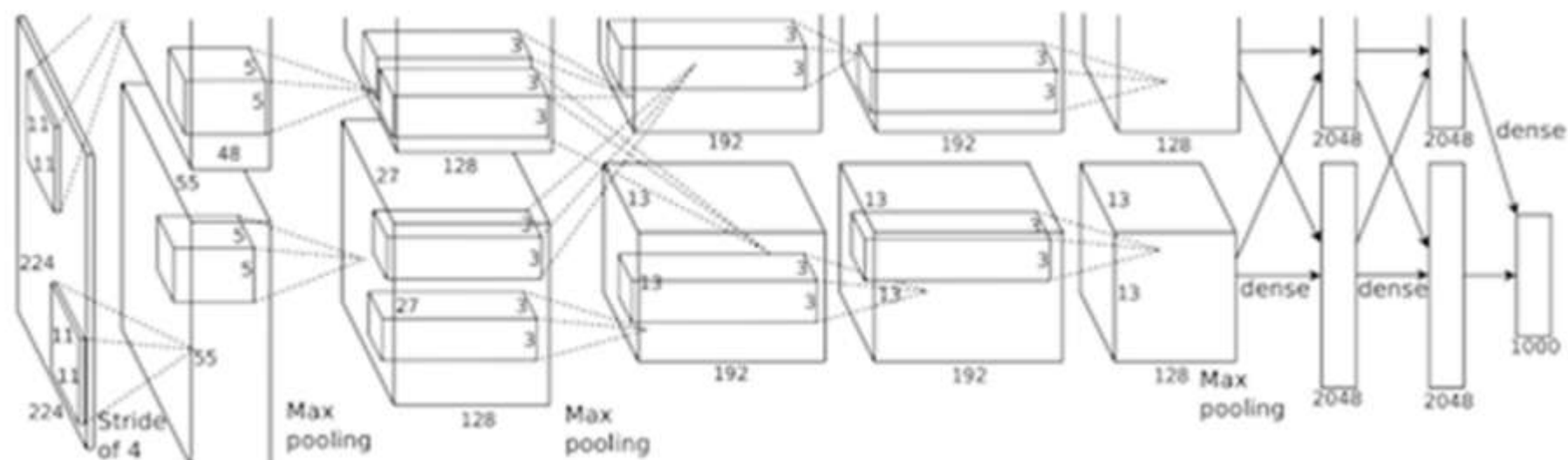
Resnet: 152层的结构，在8台GPU机器上训练2到3周。

(ILSVRC 2015 3.6%)

AlexNet:

(2012 ILSVRC top 5 test error rate of 15.4%)

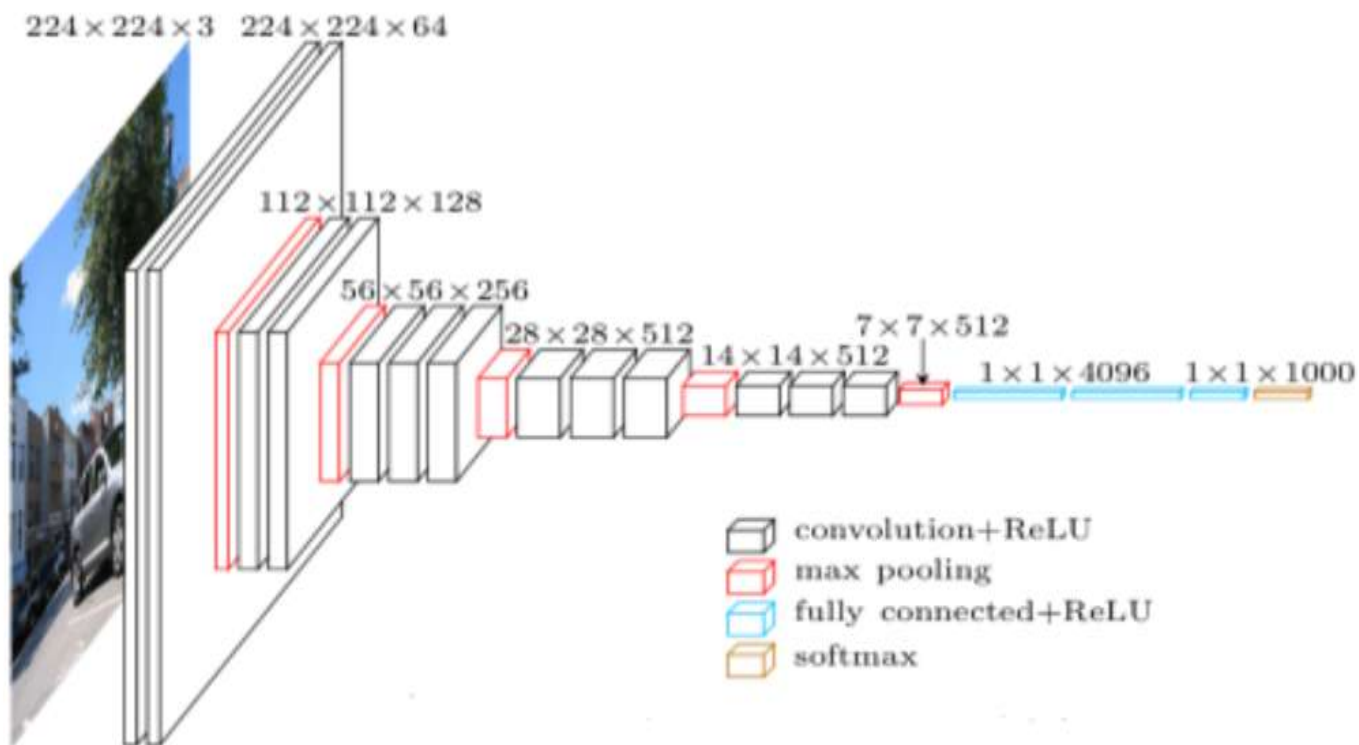
第一个成功展现出卷积神经网络潜力的网络结构。



VGGnet:

(ILSVRC 2014 7.3% error rate)

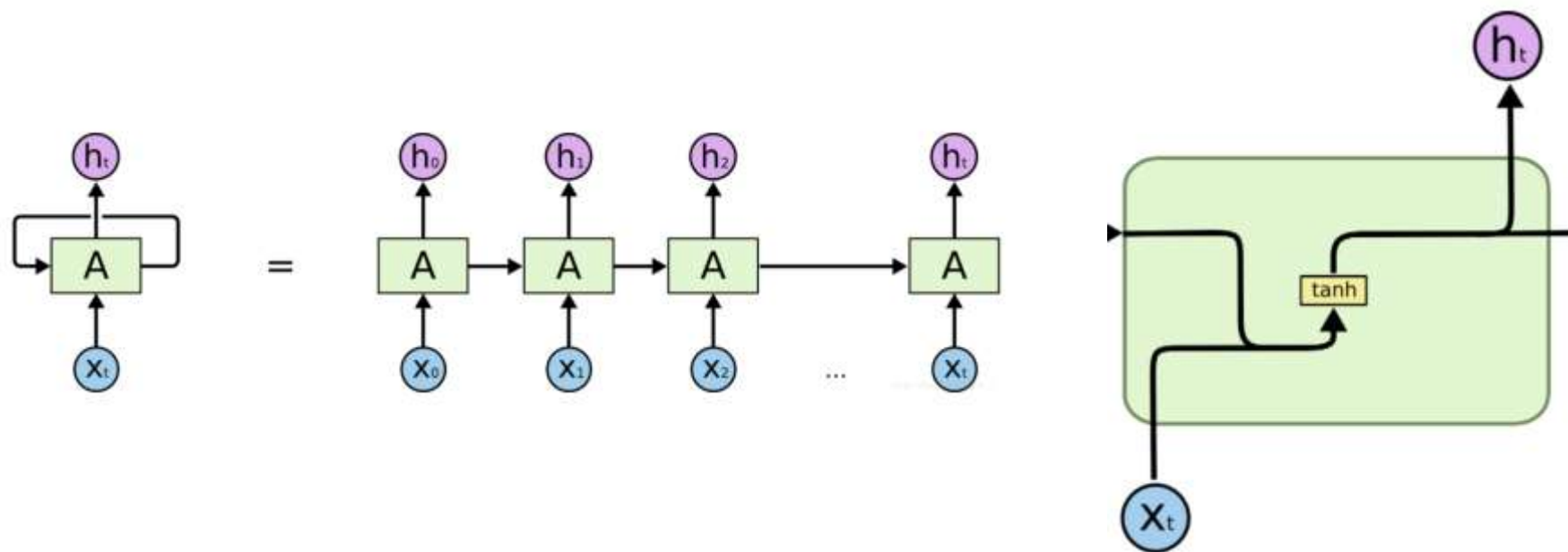
自Alexnet开始，神经网络便渐渐流行起来，开始各种尝试，最经典的是VGGnet



深度学习

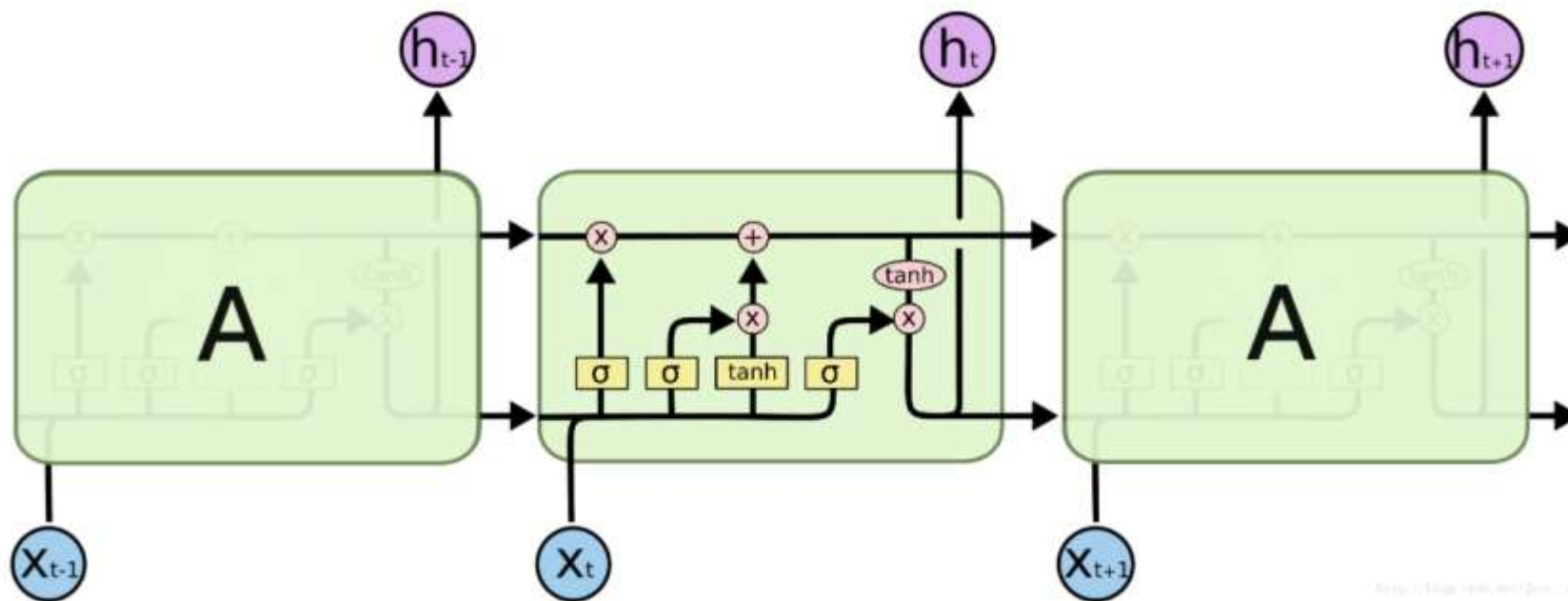
□ 循环神经网络RNN - Recurrent Neural Network

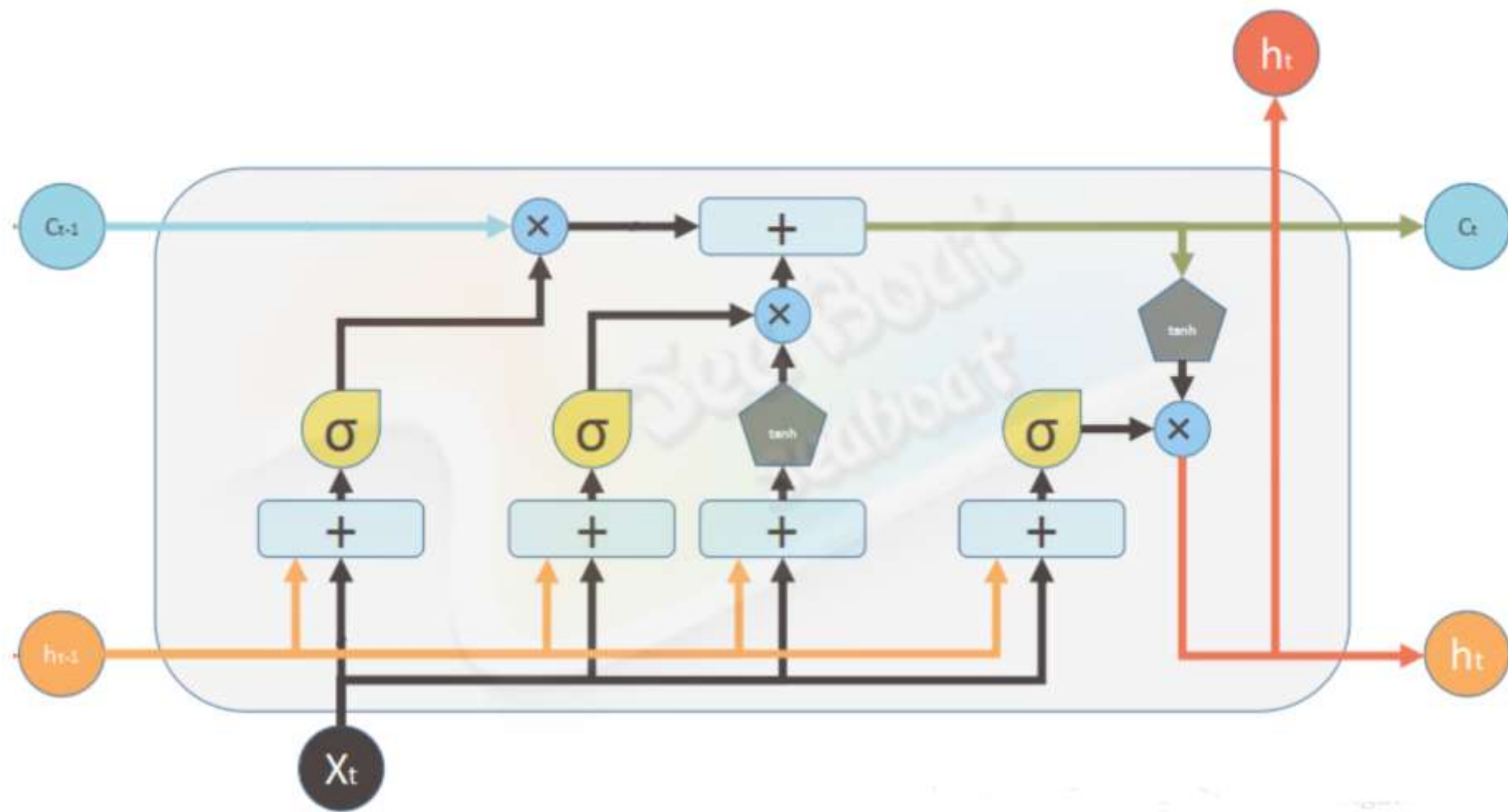
- 网络中的循环结构使得某个时刻的状态能够传到下一个时刻。
(因为当前时刻的状态会作为下一时刻输入的一部分)。从RNNs 的结构很容易理解到它是和序列信息相关的。这种结构似乎生来就是为了解决序列相关问题的,如语音识别、翻译等。

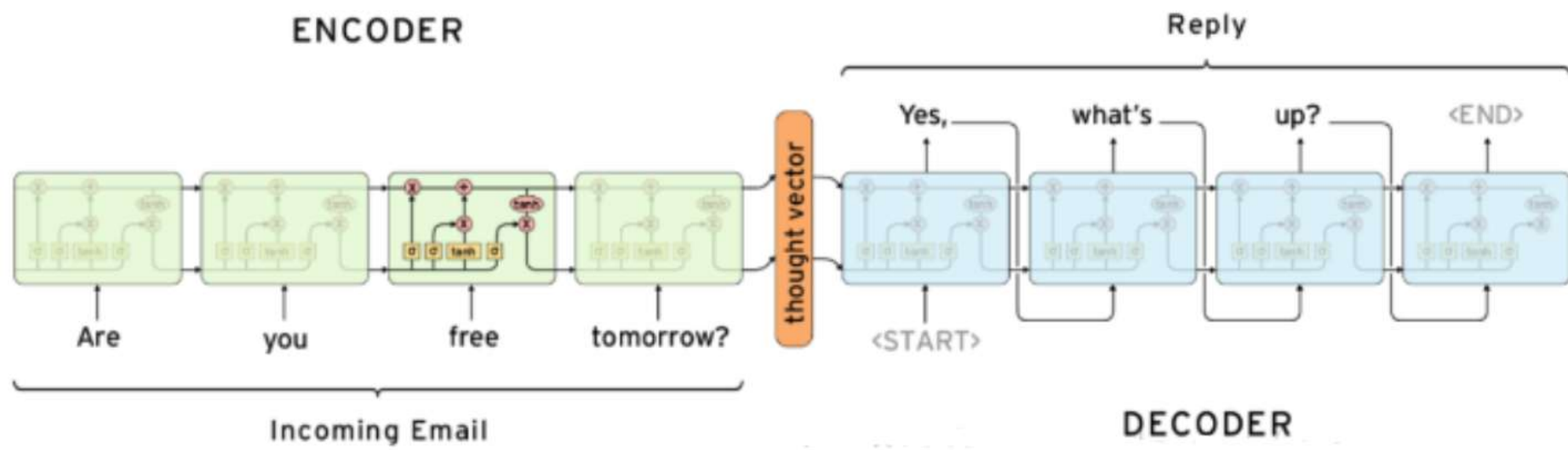


深度学习

- 长短期记忆网络 LSTMs (Long Short Term Memory networks)
- 本质就是能够记住很长时期内的信息，包括：遗忘门、传入门、输出门。
- 自然语言处理利器。



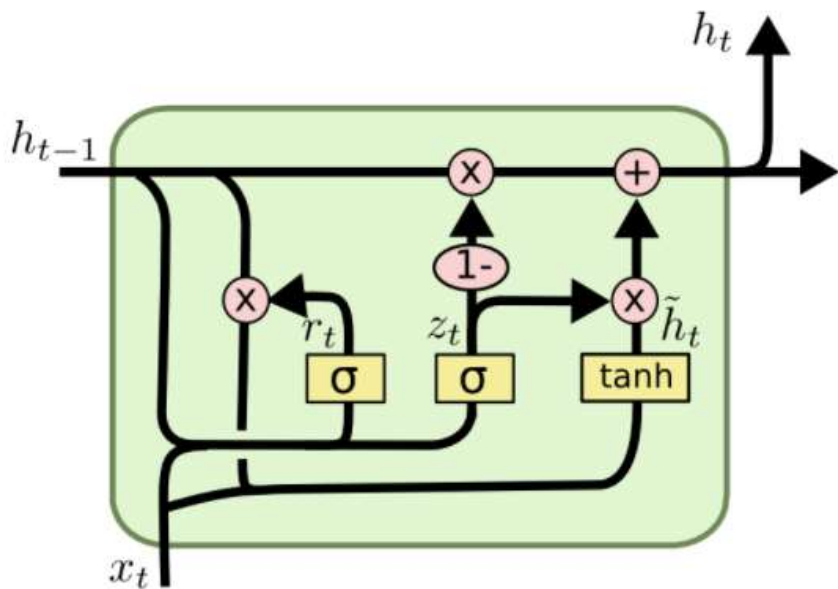




深度学习

▣ GRU网络 Gated Recurrent Unit

▣ LSTM 的变种，只有两个门：重置门（reset gate）和更新门（update gate）



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

聊天机器人

Q: i like you.

A: You're not.

Q: i really like you.

A: Yeah?

Q: thank you.

A: I'm not sure.

Q: jack chen

A: Hi.

Q: what time is it?

A: Two months.

Q: what's the time?

A: I do n't know.

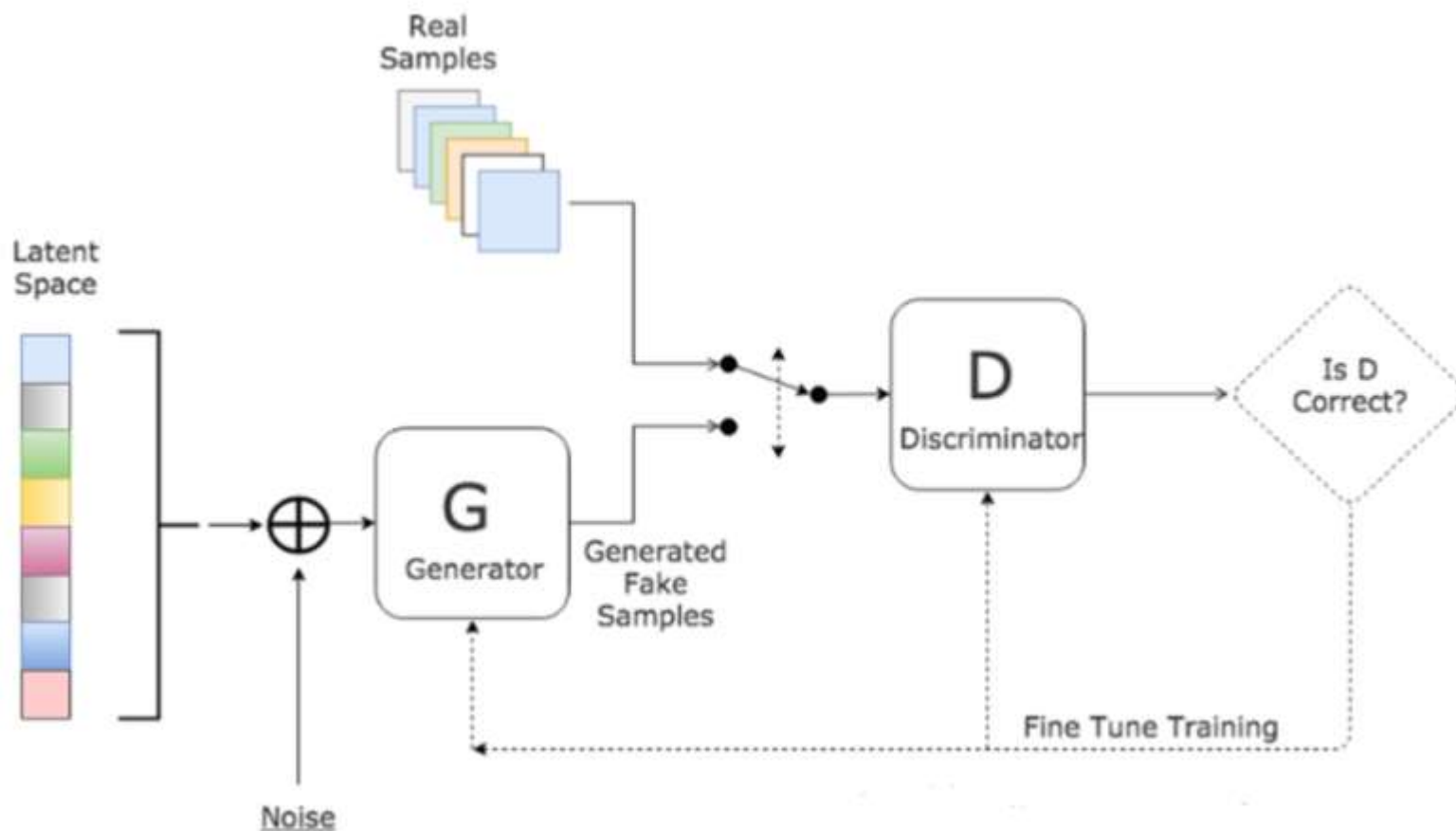
Q: you are cute.

A: I'm not.

深度学习

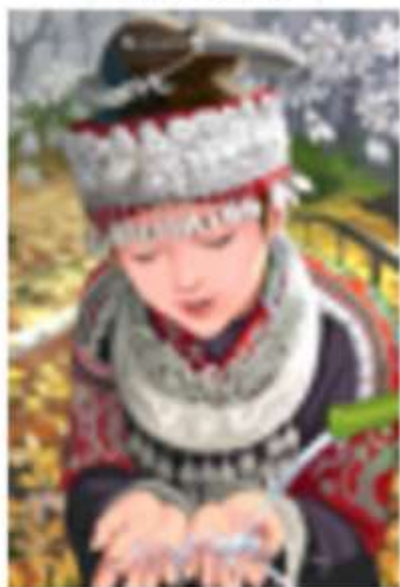
□ GAN网络 Generative Adversarial Networks

□ GAN是通过学习样本的特征自己生成样本，而不是单纯的记忆样本

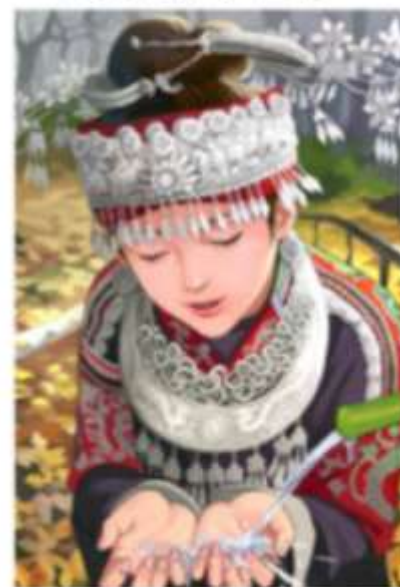




模糊图像



高分辨率图像





**2014
GAN**



**2015
DCGAN**



**2016
BEGAN**



**2017
Pro GAN**





文本到图像

根据文字：一只黑色冠冕和黄色喙的白色的鸟



LSTM提出者

Jürgen Schmidhuber



深度学习

□ 理解深度学习

- “特征工程” VS “特征学习” 或者 “表示学习”
- 特征工程由人类专家根据现实任务来设计，特征提取与识别是单独的两个阶段；



- 特征学习通过深度学习技术自动产生有益于分类的特征，是一个端到端的学习框架。



深度学习

□ 深度学习常用软件包

- Tensorflow

<https://www.tensorflow.org/>

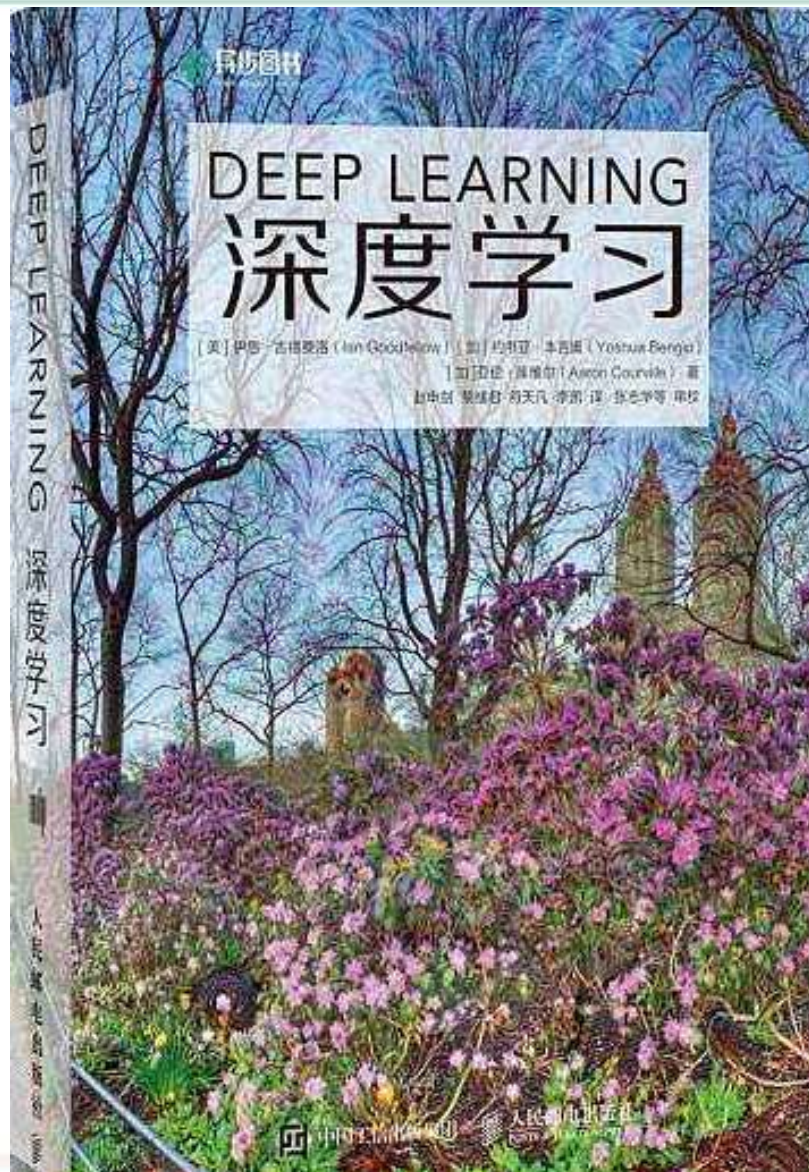
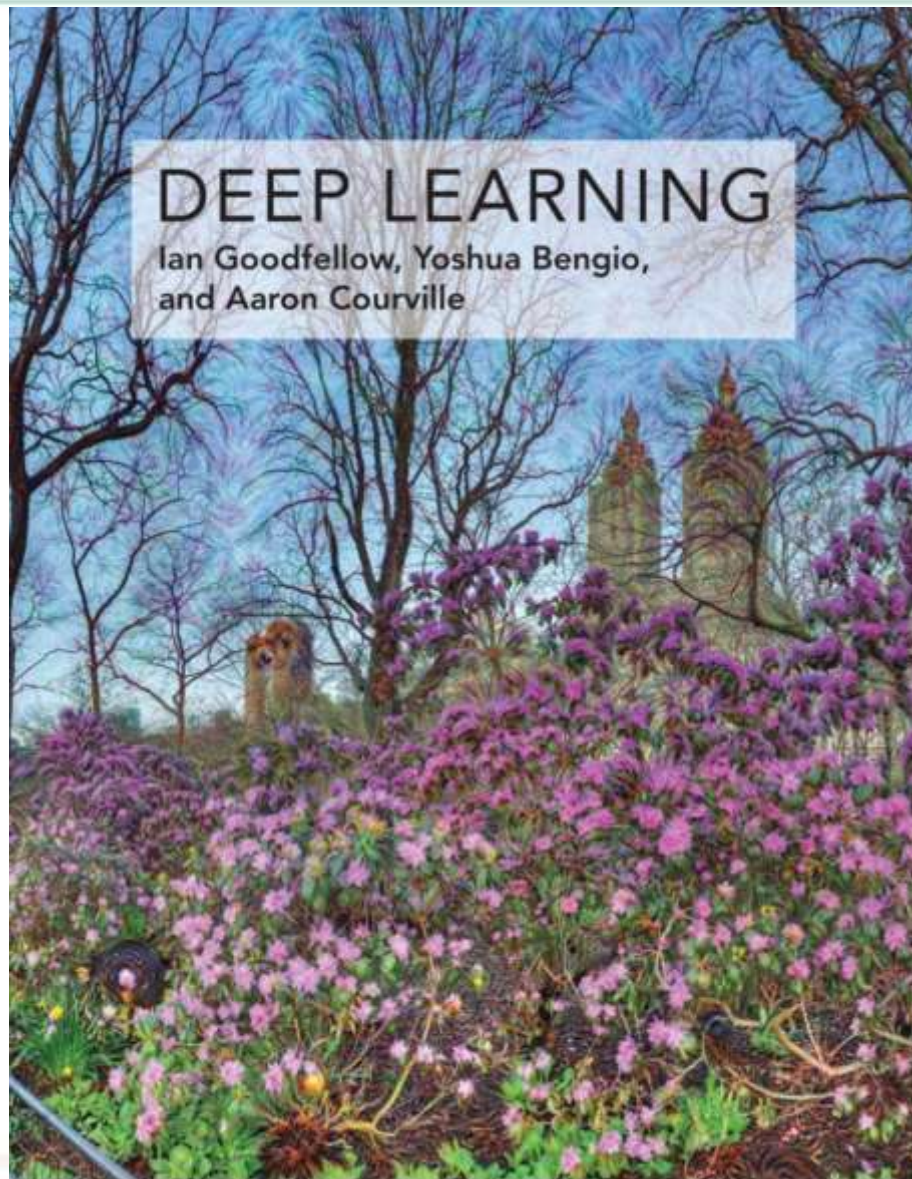
- PyTorch

<https://Pytorch.org/>

□ 神经网络基础教材

<http://hagan.okstate.edu/>

教材



阅读材料

□ 主流学术期刊

- Neural Computation
- Neural Networks
- IEEE Transactions on Neural Networks and Learning Systems

□ 国际会议

- 国际神经信息处理系统会议 (NIPS)
- 国际神经网络联合会议 (IJCNN)

□ 区域性国际会议

- 欧洲神经网络会议 (ICANN)
- 亚太神经网络会议 (ICONIP)



THANKS
祝学习快乐!